# Dual-Time-Scale Framework for Joint Optimization of Service Caching and UAV Trajectory Based on Self-Attention Deep Reinforcement Learning

Xuewei Zhang, *Member, IEEE*, Jiantao Li, Yuan Ren, *Member, IEEE*, Fan Jiang, Junxuan Wang, and Jie Zeng, *Senior Member, IEEE*

***Abstract*—Nowadays, unmanned aerial vehicle (UAV)-assisted mobile edge computing (MEC) has been recognized as a promising technique for flexibly handling computation tasks in 5G advanced and 6G networks. This article investigates the joint optimization of service caching and computation offloading within a dual time-scale framework. We maximize the caching utility and minimize the task processing delay by jointly optimizing service caching policies, UAV flight trajectory, and computation offloading decisions. Specifically, for the long-term problem, we use the latent Dirichlet allocation (LDA) model to predict user preferences and propose a Lagrangian dual decomposition-based algorithm. For the short-term problem, a self-attention-based multiagent proximal policy optimization (MAPPO) algorithm is designed. Under the centralized training with decentralized execution (CTDE) framework, this algorithm integrates a multihead self-attention mechanism with curriculum learning. Each UAV is regarded as an agent, and a self-attention encoder (SAE) is integrated at the front-end of each actor network. This enables the agent to dynamically capture the relative importance between itself and all users, and context-aware features are extracted to make more intelligent and trajectory designs. Through extensive simulation experiments, the long-term algorithm yields the performance improvements of 76.5% in cache hit rate and 66% in caching utility, as compared to the second-best baseline. In dynamic scenarios, the short-term algorithm achieves a 16% reduction in total processing latency with respect to the proximal policy optimization (PPO) policy.**

***Index Terms*—Mobile edge computing (MEC), reinforcement learning, self-attention mechanism, unmanned aerial vehicle (UAV).**

## I. Introduction

DURING the past two decades, the world has experienced more than 7000 major natural disasters [1]. After the disasters, terrestrial communication facilities may be completely destroyed [2], and complex ground conditions make it difficult to locate injured victims. To effectively deal with delay-sensitive tasks in disaster zones, mobile edge computing (MEC) has emerged as a promising approach by precaching service data or programs and providing computation services at the network edge [3], [4], [5], [6], [7]. The key idea behind MEC is to first deploy edge servers at local access points (APs) or cellular base stations (BSs), and then the tasks can be offloaded from user equipments (UEs) to the servers for remote executions [8].

As we know, infrastructure-based MEC deployment is typically costly, and ground facilities are often inaccessible in postdisaster areas. In this case, unmanned aerial vehicles (UAVs) have shown great potential in providing on-demand services for ground users [9], [10], [11], [12]. By leveraging the unique advantages of UAVs, such as flexible 3-D mobility, ease of deployment, line-of-sight (LoS) connectivity, and excellent loading capacity, UAV-assisted MEC, namely, UAV-MEC, can flexibly and effectively provide computation services to dispersed ground users when infrastructure-based MEC servers become overloaded or unavailable [13], [14], [15], [16].

In MEC scenarios, efficient task offloading is essential to ensure timely and quick task processing. In [17], the problem of service placement in an MEC-based unmanned delivery scenario was observed, with the aim of minimizing the total cost of service providers. The authors proposed a heterogeneous delivery as a service (HDaaS) framework and designed a two-stage cost-aware service placement strategy. A genetic-based algorithm and a cost-aware delivery service placement (CDSP) strategy based on a reinforcement learning (RL) algorithm were adopted in the two stages, respectively, and the optimal service combinations and placement decisions were obtained. Kazmi et al. [18] evaluated the performance of nine representative optimization algorithms for dynamic MEC service placement. This work modeled the service placement design as an NP-hard multiobjective optimization problem. The waiting and communication delays, the distance between users and servers, and the number of active servers were jointly minimized, and the load balancing across all servers was achieved. In [19], the problem of minimizing the overall delay caused by time-sensitive service placement and migration processes in autonomous vehicular networks was investigated, and a comprehensive solution considering resource allocation as well as vehicle location and trajectory prediction was obtained. Although the aforementioned studies

Received 9 March 2026; revised 5 May 2026; accepted 19 May 2026. Date of publication 25 May 2026; date of current version 27 July 2026. *(Corresponding author: Yuan Ren.)*

Xuewei Zhang, Jiantao Li, Yuan Ren, Fan Jiang, and Junxuan Wang are with the School of Communications and Information Engineering, Xi'an University of Posts and Telecommunications, Xi'an 710121, China (e-mail: zhangxw@xupt.edu.cn; lijiantao@stu.xupt.edu.cn; renyuan@xupt.edu.cn; fjiangwbc@gmail.com; wangjx@xupt.edu.cn).

Jie Zeng is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: zengjie@bit.edu.cn).

Digital Object Identifier 10.1109/JIOT.2026.3696305

2327-4662 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

utilized various algorithms to distribute MEC servers and make offloading decisions, they failed to consider diversified user request preferences, and the service hit rate could be improved if the user preferences were included.

There is some research toward UAV and MEC-enabled designs, which inspires us a lot. In a UAV-MEC system, Liu et al. [20] proposed a double deep $Q$-network and convex optimization-based algorithms to solve the two divided sub-problems, i.e., the UAV trajectory and task offloading sub-problem and the UAV flight time allocation sub-problem. The solutions were then found through an iterative optimization process. In [21], a novel online hierarchical optimization framework was introduced. At the top layer, a Lyapunov optimization framework was used to decompose the long-term problem into a series of independent single-time-slot optimizations, thereby satisfying the real-time requirements of the online framework. At the bottom layer, the coordinate descent method was employed to divide the problem within each time slot into UAV deployment, user-UAV association, task offloading, and bandwidth allocation sub-problems. Although the studies mentioned above optimized UAV trajectories and offloading strategies, they ignored feature observations, such as their relative positions to users, in dynamic and multi-UAV scenarios. Furthermore, UAVs equipped with edge servers will become more efficient and intelligent within the space–air–ground–sea integrated network (SAGSIN). In highly dynamic environments, ensuring communication security and efficient distributed computing have become key challenges. Du et al. [22] point out that distributed artificial intelligence (AI) offers an advanced paradigm for enhancing the information security and resource management performance [22]. Distributed multiagent RL (MARL) is recognized as a key enabler for dynamic task offloading and secure edge-to-cloud computing. In the specific scenario involving multi-UAV-assisted MEC, Zhang et al. [23] demonstrated that the MARL-based CTDE architecture could effectively optimize UAV-user associations, resource allocation, and system utility using the heterogeneous-agent proximal policy optimization (HAPPO) algorithm.

Typically, highly dynamic networks are confronted with significant challenges in representing spatiotemporal states. To deal with rapid changes in network topologies, Wang et al. [24] proposed to use graph-based temporal networks to enhance the feature extraction capabilities of proximal policy optimization (PPO). This inspires us that, in postdisaster UAV-MEC networks, the sparse state space and topological changes caused by intermittent user requests similarly require a feature extraction mechanism that outperforms traditional MLP. Therefore, this article adopts a multiagent PPO (MAPPO) algorithm based on the CTDE architecture and innovatively integrates a self-attention mechanism. This architecture not only addresses the catastrophic forgetting problem caused by varying user distributions but also lays a critical foundation for intelligent decision-making in security-related mechanisms, such as anti-jamming communications and location privacy protection.

To effectively address the joint optimization problem of edge caching, task offloading, and UAV trajectory designs in postdisaster scenarios, this article studies a new framework. Since content caching and task processing are conducted in the long and short time scales, respectively, we do not integrate caching utility and processing latency into a unified performance metric. Instead, we employ a dual time-scale optimization framework to decouple the cache placement problem from the task offloading problem. These two metrics are also coupled with each other. Long-term cache optimization guides the UAV to output a flight path influenced by caching decisions and user requests. Meanwhile, users' actual task requests across all time slots in the short-term layer serve as the input to the long-term prediction model. In the long-term layer, an LDA prediction model is used to forecast the service request popularity. Then, we proceed to solve the long-term caching utility maximization problem by using the Lagrangian dual decomposition algorithm. In the short-term layer, an MAPPO algorithm with a self-attention mechanism is employed to optimize the UAV flight trajectories and task offloading strategies to minimize the total task processing delay. The main contributions of this article are summarized as follows.

1) In the UAV-MEC system studied in this work, cache optimization at the long-term layer, as well as task offloading and UAV trajectory planning at the short-term layer, are highly coupled. Specifically, the cache configuration based on service popularity enables a novel content-aware trajectory design, where the UAV path is driven by cached contents and user demands, and then the task offloading delay is minimized while a high cache hit rate is guaranteed. On the other hand, actual task requests from each time slot in the short time scale are fed into the long-term layer as historical requests to predict service popularity. Considering the fact that the cache placement and task offloading are conducted in different time scales, we then propose a dual time-scale joint optimization framework to investigate the aforementioned issues.
2) In the long-term layer, we first introduce the LDA model to predict the personalized request popularity of each user. Then, an efficient iterative algorithm based on Lagrangian dual decomposition is proposed to solve the caching utility maximization problem. This algorithm updates the Lagrange multipliers by combining the Adam optimizer with adaptive learning rates and solves the problem using a greedy strategy-based heuristic approach. This approach achieves the near-optimal caching utility while bringing a lower computational complexity.
3) For the short-term layer, we propose a two-stage decoupled execution framework to address highly complex joint optimization problems. Within this framework, an enhanced MAPPO algorithm integrating a multihead self-attention encoder (SAE) is employed for continuous multi-UAV trajectory planning, while a greedy strategy is designed to solve discrete task offloading and user association problems. The SAE embedded into the front end of the MAPPO can act as a dynamic noise filter to assign near-zero attention weights to inactive users.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

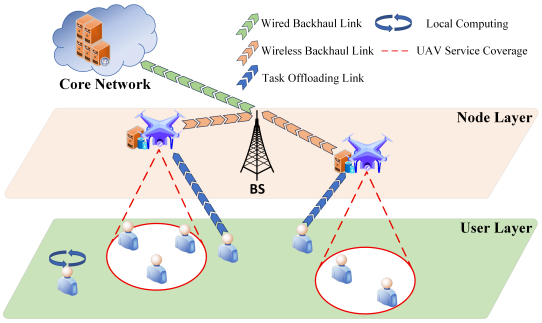



Fig. 1. System model.

This accelerates the convergence speed and enhances its stability. In addition, the SAE inherently possesses the characteristic of permutation invariance. The combination of MAPPO and SAE ensures that the decisions made in distributed actor networks no longer rely on fixed indices, and the user priorities can be evaluated based on relative contextual features. The SAE can also combine physical space with service caching states, and implicit collaborations among multiple UAVs are facilitated.

4) Comprehensive simulation experiments are conducted under both static and dynamic scenarios. As compared to multiple traditional optimization methods and RL-based baselines, results show that our method exhibits superior caching utility, task processing delay, and system robustness with fixed and changing user requests.

The remainder of this article is organized as follows. Section II presents the system model. Section III formulates the optimization problem of interest for the long-term and short-term layers. Sections IV and V specify the proposed algorithms for solving the problems in two layers, respectively. Section VI presents simulation experiments and gives corresponding discussions. Finally, Section VII concludes this article.

## II. System Model

### A. System Overview

As shown in Fig. 1, we investigate a UAV-MEC system with multiple UAVs and ground users. Each UAV is equipped with an edge server and acts as an aerial BS to serve multiple ground users. A user's computation task, such as processing an image or the video data collected from disaster areas, can either be executed locally or offloaded to its serving UAV. Due to the limited storage capacity of each UAV, only a subset of programs for conducting user tasks is cached. If the service program needed by a certain task is not locally available, the requested program needs to be retrieved from the data center in the core network via a wireless/wired backhaul link, incurring additional transmission latency.

### B. Network Model

In the considered UAV-MEC system, we denote the set of ground users by $\mathcal{U} = \{u_1, \ldots, u_i, \ldots, u_U\}$. The location of user $u_i \in \mathcal{U}$ is given as $\mathbf{pos}_{u_i} = [x_{u_i}, y_{u_i}]^T$. Let set $\mathcal{N} = \{n_1, \ldots, n_j, \ldots, n_N\}$ be the set of all UAVs. UAV $n_j \in \mathcal{N}$ operates at a fixed altitude of $H_{n_j}$ [25], [26], [27]. With a fixed flight altitude, the horizontal and vertical trajectory designs are decoupled, which can reduce the overall computational complexity of trajectory designs and enable practical implementations. The horizontal location of UAV $n_j \in \mathcal{N}$ at time slot $t$ is denoted by $\mathbf{pos}_{n_j}(t) = [x_{n_j}(t), y_{n_j}(t)]^T$. The set of users served by UAV $n_j$ at time slot $t$ is represented by $\Omega_{n_j}(t)$. We assume UAV collisions are avoided, since each UAV operates at a different altitude.

### C. Communication Model

In the considered air-to-ground communication scenario, the channel quality between the UAV and the user is significantly influenced by the UAV's altitude and its elevation angle. Therefore, we adopt the widely used probabilistic LoS model [28]. The elevation angle between UAV $n_j$ and user $u_i$ at time slot $t$, denoted by $\theta_{n_j,u_i}(t)$, is calculated as

$$\theta_{n_j,u_i}(t) = \arctan\left(\frac{H_{n_j}}{d^{\text{horz}}_{n_j,u_i}(t)}\right) \tag{1}$$

where $d^{\text{horz}}_{n_j,u_i}(t) = \|\mathbf{pos}_{u_i} - \mathbf{pos}_{n_j}(t)\|$. The probability of having an LoS link, denoted as $P_{\text{LoS}}$, is given by [28]

$$P_{\text{LoS}}(\theta_{n_j,u_i}(t)) = \left(1 + a\exp\left(-b\left[\theta_{n_j,u_i}(t)\frac{180}{\pi} - a\right]\right)\right)^{-1} \tag{2}$$

where $a$ and $b$ are environment-related constants. Correspondingly, the probability of having a non-LoS (NLoS) link is represented as

$$P_{\text{NLoS}} = 1 - P_{\text{LoS}}. \tag{3}$$

In the LoS case, the path loss (PL) is modeled as the free-space PL, shown as follows:

$$PL_{\text{LoS}}(t) = 20\lg(d_{n_j,u_i}(t)) + 20\lg(f_c) + 20\lg\left(\frac{4\pi}{c}\right) \tag{4}$$

where $d_{n_j,u_i}(t) = (H^2_{n_j} + d^{\text{horz}}_{n_j,u_i}(t)^2)^{1/2}$ is the Euclidean distance between UAV $n_j$ and user $u_i$, $f_c$ is the carrier frequency, and $c$ denotes the light speed. For the NLoS case, an additional loss term, $\eta_{\text{NLoS}}$, is introduced to account for the signal attenuation from obstacles. The NLoS PL is finally obtained as

$$PL_{\text{NLoS}}(t) = PL_{\text{LoS}}(t) + \eta_{\text{NLoS}}. \tag{5}$$

From the above, the PL over LoS and NLoS cases is calculated as

$$PL_{\text{avg}}(t) = P_{\text{LoS}}(t) \cdot PL_{\text{LoS}}(t) + P_{\text{NLoS}}(t) \cdot PL_{\text{NLoS}}(t). \tag{6}$$

The channel gain is given by

$$\left|h_{n_j,u_i}(t)\right|^2 = (10^{PL_{\text{avg}}(t)/10})^{-1}. \tag{7}$$

To eliminate intra-UAV interference, we adopt an orthogonal frequency division multiple access (OFDMA) approach [29]. In this scheme, the spectrum resource allocated to user $u_i$ served by UAV $n_j$ is $B^{u_i}_{n_j}$. The signal-to-interference-plus-noise

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

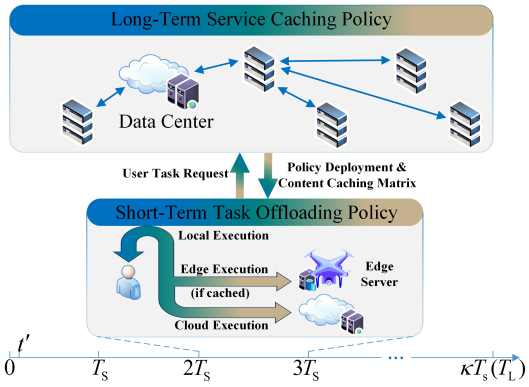



Fig. 2. Dual time-scale model for collaborative service caching and task offloading framework.

ratio (SINR) for user $u_i$ served by UAV $n_j$ at time slot $t$ is expressed as

$$\mathrm{SINR}_{n_j,u_i}(t) = \frac{p_{n_j}\left|h_{n_j,u_i}(t)\right|^2}{\sum_{n_k\in\mathcal{N},n_k\neq\{n_j\}} p_{n_k}\left|h_{n_k,u_i}(t)\right|^2+\sigma^2} \tag{8}$$

where $p_{n_j}$ is the transmit power of UAV $n_j$. The first term in the denominator represents the intercell interference from other UAVs. The second term $\sigma^2 = N_0 B_{n_j}^{u_i}$ denotes the noise power, where $N_0$ is the spectral density of the noise power. Then, the achievable transmission rate for user $u_i$ to UAV $n_j$ at time slot $t$ is given by

$$R_{n_j,u_i}(t) = B_{n_j}^{u_i}\log_2\left(1+\mathrm{SINR}_{n_j,u_i}(t)\right). \tag{9}$$

If the cache miss case happens, we assume that the transmission rate of the backhaul link from the cloud center to the UAV, denoted by $R_{\mathrm{BH}}$, is defined as the average effective throughput over the time slot [30].

### D. Service Caching Model

In general, the service caching policy is performed on a large time scale. When designing caching decisions, our goal is to prioritize using edge servers to process tasks. The caching and computing capacities allocated to UAV $n_j$ are denoted by $S_{n_j}$ and $C_{n_j}$, respectively. In our simulation experiments, for simplicity but without loss of generality, we set $S_{n_j} = S$ and $C_{n_j} = C$ for all UAVs. Considering the time-varying and periodic nature of the system, the processes of service caching and task loading are conducted in long-term and short-term time periods, with the durations of $T_{\mathrm{L}}$ ($\kappa T_{\mathrm{S}}$) and $T_{\mathrm{S}}$, respectively, as shown in Fig. 2. All indices of time slots are collected in the set $\mathcal{T}$.

We assume there are a total of $K$ distinct programs available in the cloud, and the programs correspond to $R$ classes. The caching storage and computational resources required to execute program $k$ are denoted by $s_k$ and $f_k$, respectively.

At the beginning of the long-term time slot, i.e., $t'$, a binary variable $x_{k,n_i}(t') \in \{0,1\}$ is introduced to denote the caching decision for program $k$ on edge node $n_j$ at time slot $t'$. Specifically, $x_{k,n_i}(t') = 1$ if program $k$ is cached on node $n_j$; otherwise, we have $x_{k,n_i}(t') = 0$. Consequently, the caching policy of node $n_j$ at time slot $t'$ is represented by $\mathbf{x}_{n_j}(t') = [x_{1,n_j}(t'),\ldots,x_{K,n_j}(t')]$.

### E. Task Offloading Model

The task offloading strategy is conducted on a short time-scale. This short-term optimization occurs on a slot-by-slot basis. We assume that user $u_i$ generates a single computation task, i.e., $M_{u_i}$, at the beginning of each time slot. Task $M_{u_i}$ is characterized by the following parameters.

1) Data size $D_{u_i}$ (in bits).
2) Total computation workload, calculated as $W_{u_i} = D_{u_i}X_{u_i}$ (in CPU cycles), where $X_{u_i}$ is the number of CPU cycles required to process a single bit of data.
3) Program required for conducting the current task, written as $K_{u_i}$.

The proposed optimization framework operates on a two-time-scale basis. First, at the beginning of each long time slot, a long-term caching policy is designed to determine the program caching decisions. Then, for each time slot, a short-term task offloading scheme is determined. The process within each time slot is performed in three phases, namely, data uploading, task processing, and result downloading. We focus on computation-intensive tasks, and it is reasonably assumed that the data size of the output result is negligible compared to that of the input data. Consequently, the downlink transmission delay is not explicitly included in our analysis [31].

There are three methods for conducting computation tasks: 1) local execution in UE; 2) edge execution by offloading the task to the edge node if the needed program is cached; and 3) cloud processing if the program is not locally accessible. The task processing delay for each mode consists of communication and computation delays.

The uplink communication delay for offloading task $M_{u_i}$ to node $n_j$ during time slot $t$ is defined as

$$T_{M_{u_i},n_j}^{ul}(t) = \frac{D_{u_i}(t)}{r_{n_j,u_i}(t)}. \tag{10}$$

Let $f_{u_i}(t)$, $f_{n_j,u_i}(t)$, and $f_c(t)$ be the computation capabilities (in CPU hz) owned by user $u_i$, allocated to user $u_i$ by node $n_j$, and reserved for cloud center, respectively. The total computation capacity $C_{n_j}$ of UAV $n_j$ is equally divided among its associated users $\Omega_{n_j}(t)$. Thus, $f_{n_j,u_i}(t) = C_{n_j}/|\Omega_{n_j}(t)|$. The corresponding computation delays of the three methods for conducting task $M_{u_i}$ are then given as follows.

1) *Local Processing Delay:* $T_{u_i}^{\mathrm{lo}}(t) = (W_{u_i}(t))/(f_{u_i}(t))$.
2) *Edge Processing Delay:* $T_{n_j,u_i}^{\mathrm{of}}(t) = (W_{u_i}(t))/(f_{n_j,u_i}(t))$.
3) *Cloud Processing Delay:* $T_{n_j,u_i}^{\mathrm{cl}}(t) = (W_{u_i}(t))/(f_c(t))$.

We define a binary variable $a_{n_j,u_i}(t) \in \{0,1\}$ to indicate whether user $u_i$ is associated with node $n_j$ at time slot $t$. Specifically, $a_{n_j,u_i}(t) = 1$ holds, if user $u_i$ is associated with $n_j$ at time slot $t$, and $a_{n_j,u_i}(t) = 0$ otherwise. Then, the UAV-user association strategy at time slot $t$ is represented by $\mathbf{A}(t) = [a_{n_j,u_i}(t)]_{U\times N}$.

## III. Problem Formulation

### A. Long-Term Optimization

The primary goal of long-term optimization is to design the service caching policy, with the aim of maximizing the system caching utility. Before formulating the long-term optimization problem, we define the following decision variables.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

1) The decision variable for caching program $k$ at node $n_j$ is $x_{k,n_j}(t') \in \{0,1\}$. Define $\mathbf{X}(t') = [x_{k,n_j}(t')]_{K\times N}$.
2) Auxiliary variables $d^+_{k,n_j}(t')$ and $d^-_{k,n_j}(t')$ are introduced to track the dynamics of caching status. Specifically, $d^+_{k,n_j}(t') = 1$ denotes that program $k$ is added to the cache of node $n_j$ at time slot $t'$, while $d^-_{k,n_j}(t') = 1$ indicates this program is removed. The two variables satisfy $d^+_{k,n_j}(t') = \max\{x_{k,n_j}(t') - x_{k,n_j}(t'-T_{\mathrm{L}}), 0\}$, $d^-_{k,n_j}(t') = \max\{x_{k,n_j}(t'-T_{\mathrm{L}}) - x_{k,n_j}(t'), 0\}$.

In this article, following the profit-maximization framework widely used in the literature on cache management, we plan to achieve a preferable tradeoff between content popularity and resource consumption, and the caching utility of the system is defined as [32]

$$CU = U_{\mathrm{be}} - C_{\mathrm{st}} - C_{\mathrm{re}} \tag{11}$$

where $U_{\mathrm{be}}$, $C_{\mathrm{st}}$, and $C_{\mathrm{re}}$ represent the caching benefit, total storage cost, and cache replacement cost, respectively.

Specifically, the caching benefit $U_{\mathrm{be}}$ is shown as

$$U_{\mathrm{be}} = \sum_{k\in\mathcal{K}} \left( l(k) \sum_{n_j\in\mathcal{N}} x_{k,n_j}(t') \right) \tag{12}$$

where $l(k)$ represents the request popularity of program $k$, calculated as

$$l(k) = \gamma_1 P_k + \gamma_2 N_k. \tag{13}$$

In (13), $P_k$ and $N_k$ denote the predicted and historical request probabilities for program $k$, respectively, and $\gamma_1$ and $\gamma_2$ are weighting coefficients to balance the importance between the prediction result and recent trend. $P_k$ is obtained as

$$P_k = \sum_{u_i\in\mathcal{U}} P(w_k|D_{u_i}) \tag{14}$$

where $P(w_k|D_{u_i})$ is the user-specific popularity of program $k$ for user $u_i$.

The second term in (11) is formulated as

$$C_{\mathrm{st}}(t') = \alpha \sum_{k\in\mathcal{K}} \sum_{n_j\in\mathcal{N}} \mathrm{Cost}(x_{k,n_j}(t')). \tag{15}$$

In the expression for $C_{\mathrm{st}}$, $\alpha$ denotes a weighting factor and $\mathrm{Cost}(x_{k,n_j}(t'))$ gives the cost of caching program $k$ on node $n_j$ at time slot $t'$, as given by

$$\mathrm{Cost}(x_{k,n_j}(t')) = \zeta_k s_k x_{k,n_j}(t') \tag{16}$$

where $\zeta_k$ is the caching cost coefficient per bit.

Scaled by a weighting factor $\beta$, the total replacement cost is given by

$$C_{\mathrm{re}}(t') = \beta \sum_{n_j\in\mathcal{N}} y_{n_j}(t') \tag{17}$$

where $y_{n_j}(t')$ denotes the scheduling overhead incurred by the number of cache updates for node $n_j$ at time slot $t'$. It is defined as

$$y_{n_j}(t') = \sum_{k\in\mathcal{K}} \left( d^+_{k,n_j}(t') + d^-_{k,n_j}(t') \right). \tag{18}$$

From the above, the long-term optimization problem is formulated as

$$\max_{\mathbf{X}(t')} \ CU \tag{19a}$$

$$\text{s.t.} \ \sum_{k\in\mathcal{K}} s_k x_{k,n_j}(t') \le S_{n_j} \tag{19b}$$

$$d^+_{k,n_j}(t') = \max\{x_{k,n_j}(t') - x_{k,n_j}(t'-T_{\mathrm{L}}), 0\} \tag{19c}$$

$$d^-_{k,n_j}(t') = \max\{x_{k,n_j}(t'-T_{\mathrm{L}}) - x_{k,n_j}(t'), 0\} \tag{19d}$$

$$x_{k,n_j}(t') \in \{0,1\}. \tag{19e}$$

Constraint (19b) is the caching capacity constraint, ensuring that the total storage space occupied by $n_j$ will not exceed its maximum. Equations (19c) and (19d) give the transformation relationship between decision-making variables across adjacent time slots. Finally, (19e) gives the feasible range of the caching decision variable.

### B. Short-Term Optimization

The short-term optimization is operated on a small timescale. Based on the known caching strategy, it dynamically optimizes the flight trajectories of all UAVs. $\ell_{u_i}(t)$, $e_{n_j,u_i}(t)$, and $c_{n_j,u_i}(t) \in \{0,1\}$ are defined as decision variables for local, edge, and cloud offloading, respectively. We define the following matrices $\mathbf{L} = [\ell_{u_i}(t)]_U$, $\mathbf{E} = [e_{n_j,u_i}(t)]_{U\times N}$, $\mathbf{C} = [c_{n_j,u_i}(t)]_{U\times N}$ and $\mathbf{P} = [\mathbf{pos}_{n_j}(t)]_N$. The total task processing delay is then obtained as

$$\begin{aligned} XU = \sum_{t\in\mathcal{T}} \Bigg( \max_{u_i\in\mathcal{U}} \Bigg[ & \ell_{u_i}(t) T^{\mathrm{lo}}_{u_i}(t) + \sum_{n_j\in\mathcal{N}} e_{n_j,u_i}(t) T^{\mathrm{of}}_{n_j,u_i}(t) \\ & + \sum_{n_j\in\mathcal{N}} c_{n_j,u_i}(t) T^{\mathrm{cl}}_{n_j,u_i}(t) \Bigg] + \sum_{n_j\in\mathcal{N}} \sum_{u_i\in\mathcal{U}} \Bigg[ (e_{n_j,u_i}(t) \\ & + c_{n_j,u_i}(t)) T^{\mathrm{ul}}_{M_{u_i},n_j}(t) + c_{n_j,u_i}(t) \frac{D_{u_i}(t)}{R_{\mathrm{BH}}} \Bigg]. \end{aligned} \tag{20}$$

With the aim of minimizing the task processing delay, the short-term optimization problem is established as

$$\min_{\substack{\mathbf{P},\mathbf{A},\\ \mathbf{L},\mathbf{E},\mathbf{C}}} \ XU \tag{21a}$$

$$\text{s.t.} \ \left\| \mathbf{pos}_m(t) - \mathbf{pos}_m(t-1) \right\| \le D_{\max} \cdot \Delta t \tag{21b}$$

$$\left\| \mathbf{pos}_{u_i} - \mathbf{pos}_m(t) \right\| \le d_{\max} \tag{21c}$$

$$e_{n_j,u_i}(t) \le a_{n_j,u_i}(t), \quad c_{n_j,u_i}(t) \le a_{n_j,u_i}(t) \tag{21d}$$

$$f_{n_j,u_i}(t) \ge 0, \quad \sum_{u_i\in\mathcal{U}} f_{n_j,u_i}(t) \le C_{n_j} \tag{21e}$$

$$\sum_{n_j\in\mathcal{N}} a_{n_j,u_i}(t) \le 1 \tag{21f}$$

$$\ell_{u_i}(t) + \sum_{n_j\in\mathcal{N}} e_{n_j,u_i}(t) + \sum_{n_j\in\mathcal{N}} c_{n_j,u_i}(t) = 1 \tag{21g}$$

$$a_{n_j,u_i}(t), \quad \ell_{u_i}(t), \ e_{n_j,u_i}(t), \ c_{n_j,u_i}(t) \in \{0,1\}. \tag{21h}$$

Constrain (21b) gives the distance limitation for UAV movement, where $D_{\max}$ is maximum speed and $\Delta t$ is the time duration for each time slot; (21c) refers to the communication

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

distance constraint between the UAV and its serving user, and $d_{\max}$ denotes the maximum UAV-user distance; (21d) guarantees that the task of $u_i$ can be conducted by UAV $n_j$, if the user-UAV association between them is established, or by the cloud center; (21e) accounts for the practical constraint for the computation resource allocated to each user; (21f) ensures that a single user can at most be associated with one edge server; (21g) gives the selection restriction of task processing mode; and constraint (21h) gives the feasible ranges of three binary variables. These constraints are essentially guaranteed by boundary operations in the action space and the discrete logic of stage 2.

## IV. Proposed Scheme for the Long-Term Optimization Problem

In this section, we aim to solve the long-term optimization problem.

### A. LDA Prediction Model

The rationale for employing the LDA model to obtain user preferences stems from the inherent structural similarity regarding generating documents and service requests. Specifically, we treat the request of each user as a document, and each service requested by users is regarded as a word. Users' latent interests are modeled as topics. A document is generated by mixing multiple topics, and then the specific word is generated. A user's request history is similarly driven by its underlying interest distribution. The advantage of the LDA model lies in its ability to infer these latent interest distributions from sparse historical data. This enables us to predict the probability of a user requesting a specific service in the future by leveraging the correlations captured from latent topics. Predicted service requests are generated by training an LDA model on the data gathered from previous $V$ time slots to learn user preferences [33], and the CHR and the quality of experience (QoE) of users will be significantly improved.

To model the service request, we adopted a topic modeling paradigm. Specifically, the historical requests of each user over a long-term period are recorded in a single document. The actual task request that occurred in each time slot is then considered as a word in the document.

Assume a total of $Z$ documents are collected. The $z$th document, denoted by $D_z$, is represented as a sequence composed of $T$ words, shown as

$$D_z = (w_{z,1}, w_{z,2}, \ldots, w_{z,T}). \tag{22}$$

Using variational Bayes (VB) inference, our LDA model learns the following distributions from the historical data.

1) Document-topic distribution, denoted as $\theta_{z,r}$, for document $z$, which represents the user preference for topic $r$.
2) Topic-word distribution, written as $\boldsymbol{\phi}_{r,w}$, which gives the probability of word $w$ belonging to topic $r$.

Combining the two distributions, the request probability of word $w$ in document $D_z$ is calculated by

$$P(w|D_z) = \sum_{r=1}^{R} \boldsymbol{\phi}_{r,w}\theta_{z,r}. \tag{23}$$

### B. Long-Term Optimization

The long-term optimization problem formulated in (19) is characterized as a nonlinear mixed-integer programming (MIP) problem. The long-term problem is NP-hard. When the number of service types and UAVs increases, the computational complexity of solving the problem grows exponentially.

To address this challenge, we propose a hybrid approach to find a computationally efficient solution. The Lagrangian relaxation techniques are employed to implement a dual decomposition method, which is further combined with a greedy strategy-based algorithm. The Lagrangian relaxation method is specifically chosen to handle (19b) by relaxing and integrating it as a penalty term into the objective function.

By substituting (12), (15), and (17) into (11), (19a) is simplified into

$$\sum_{k\in\mathcal{K}}\sum_{n_j\in\mathcal{N}}\Big(l(k)-\alpha(\zeta_k s_k)\Big)x_{k,n_j}(t') - \beta\sum_{k\in\mathcal{K}}\sum_{n_j\in\mathcal{N}}\Big(d^{+}_{k,n_j}(t')+d^{-}_{k,n_j}(t')\Big). \tag{24}$$

Next, we address problem (19) by introducing a series of nonnegative Lagrange multipliers, collected in the vector of $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \ldots, \lambda_{n_j}, \ldots, \lambda_{n_N}]$. Each multiplier is associated with a capacity constraint for each node. This allows us to integrate (19b) into the objective function. After some mathematical simplifications, we have

$$\sum_{k\in\mathcal{K}}\sum_{n_j\in\mathcal{N}}\Big[(l(k)-\alpha\zeta_k s_k)x_{k,n_j}(t') - \beta\left|x_{k,n_j}(t')-x_{k,n_j}(t'-T_{\mathrm{L}})\right|\Big] - \sum_{n_j\in\mathcal{N}}\lambda_{n_j}\left(\sum_{k\in\mathcal{K}}s_k x_{k,n_j}(t') - S_{n_j}\right). \tag{25}$$

The dual function is a function of $\boldsymbol{\lambda}$, defined as $g(\boldsymbol{\lambda})$. We therefore consider the following dual problem:

$$\min_{\boldsymbol{\lambda}\geq 0} g(\boldsymbol{\lambda}) \tag{26a}$$

$$\text{s.t.} \quad (19c) - (19e). \tag{26b}$$

Since $g(\boldsymbol{\lambda})$ contains binary variables, a nonzero duality gap may arise. With weak duality, we always have

$$f^* \leq g^* = \min_{\boldsymbol{\lambda}\geq 0} g(\boldsymbol{\lambda}) \tag{27}$$

where $f^*$ is the optimal value of (19a). Consequently, $g(\boldsymbol{\lambda})$ provides an upper bound of the optimal solution. After obtaining the inner-layer solution, we need to check whether the caching capacity constraint is satisfied. If the total size of cached programs exceeds the maximum capacity, the items with smaller request probability are removed from the local cache. This process repeats until the caching capacity constraint is met. Let $\widehat{\mathbf{X}}^{(i)}$ be the recovered solution and $g(\boldsymbol{\lambda}^{(i)})$ be the corresponding dual value for the $i$th outer-layer iteration. The relative duality gap is defined as

$$\Delta^{(i)}_{\mathrm{rel}} = \frac{g(\boldsymbol{\lambda}^{(i)}) - f\left(\widehat{\mathbf{X}}^{(i)}\right)}{\left|f\left(\widehat{\mathbf{X}}^{(i)}\right)\right|}. \tag{28}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

Iterations will terminate when $\Delta_{\text{rel}}^{(i)} \leq \varepsilon$ holds.

To solve the dual problem, the inner-layer maximization problem must be addressed first. Then, (25) is simplified into

$$\sum_{k\in\mathcal{K}}\sum_{n_j\in\mathcal{N}}\Big[\big(l(k)-\alpha\zeta_k s_k-\lambda_{n_j}s_k+\beta(2x_{k,n_j}(t'-T_{\text{L}})-1)\big)x_{k,n_j}(t')\Big]-\beta\sum_{k\in\mathcal{K}}\sum_{n_j\in\mathcal{N}}x_{k,n_j}(t'-T_{\text{L}})+\sum_{n_j\in\mathcal{N}}\lambda_{n_j}S_{n_j}. \tag{29}$$

The last two items are constants and can be ignored. Since (29) is linear with the decision variables, it can be decomposed into $T \times N \times K$ sub-problems. The divided sub-problem is shown as

$$\min_{\mathbf{X}}\ \big(l(k)-\alpha\zeta_k s_k-\lambda_{n_j}s_k+\beta(2x_{k,n_j}(t'-T_{\text{L}})-1)\big)x_{k,n_j}(t') \tag{30a}$$

$$\text{s.t.}\quad (19c)\ -\ (19e)\ . \tag{30b}$$

We design an alternating optimization algorithm based on dual decomposition to deal with (30). The specific alternating process is illustrated as follows.

*1) Solving the Inner-Layer Maximization Problem:* First, the LDA model and historical request frequencies are used to obtain the request probability for program $k$, i.e., $l(k)$, and a unit capacity score for UAV $n_j$ is obtained as

$$\rho_{k,n_j}=\frac{l(k)}{\mathbf{s}_k}-\alpha\zeta_k-\lambda_{n_j}\frac{\beta(2x_{k,n_j}(t'-T_{\text{L}})-1)}{\mathbf{s}_k}. \tag{31}$$

For each UAV, candidate programs are sorted in descending order of $\rho_{k,n_j}$. The programs are sequentially selected for caching until the caching space is used up. After solving the inner-layer problem, the optimal $\mathbf{X}^*(t')$ is yielded.

*2) Updating the Outer-Layer Lagrange Multipliers:* After solving the inner-layer sub-problem, the sub-gradient method is employed to update the Lagrange multipliers. The core idea is to update $\lambda_{n_j}$ according to the violation degree of the caching capacity constraint, which is evaluated by $g_{n_j}$.

Specifically, for the $i$th iteration, the caching capacity violation is calculated as

$$g_{n_j}^{(i)}=\left(\sum_{k\in\mathcal{K}}s_k x_{k,n_j}^{*(i)}(t')\right)-S_{n_j}. \tag{32}$$

If $g_{n_j}^{(i)}>0$, it signifies that the capacity occupied by the programs cached by UAV $n_j$ exceeds the upper limit, indicating an increase in $\lambda_{n_j}$. Otherwise, it shows that the capacity is not fully utilized, and the multiplier should be decreased to achieve a higher resource utilization. To deal with the slower convergence speed, the step size is adaptively changed. The strategy includes the global and local policies. More details of them are described as follows.

*a) Global strategy:* The learning rate, denoted as $\eta$, should change rapidly in the initial phase and slowly in subsequent phases. In the initial phase, $\eta$ is set to be linearly increased to its maximum. Then, $\eta$ is smoothly decreased, with the aim of ensuring that the final solution is optimal.

*b) Local strategy:* The Adam optimizer is used to update the Lagrange multipliers, and it keeps a historical record for each multiplier. Specifically, the Adam optimizer ensures fast and stable convergence by updating the first-order and the second-order moments, denoted as $m_{n_j}^{(i)}$ and $v_{n_j}^{(i)}$, respectively, shown as follows:

$$m_{n_j}^{(i)}=\beta_1 m_{n_j}^{(i-1)}+(1-\beta_1)g_{n_j}^{(i)} \tag{33}$$

$$v_{n_j}^{(i)}=\beta_2 v_{n_j}^{(i-1)}+(1-\beta_2)\left(g_{n_j}^{(i)}\right)^2 \tag{34}$$

where $\beta_1$ and $\beta_2$ are corresponding exponential decay rates.

For the server with severe constraint violation, its gradient is set to a large value, leading to a rapid accumulation of the second-order moment. Then, the optimizer reduces the step size. For the node with a slight violation, the gradient is sparse and small, resulting in a smaller second-order moment. The optimizer can increase the step size. By introducing the first-order moment, the Adam optimizer increases the multipliers for the nodes persistently, resulting in constraint violations, and thus, the violation suppression is quickly achieved.

Since the first-order and second-order moments are initialized to zero, the following bias corrections are required for these estimates:

$$\hat{m}_{n_j}^{(i)}=\frac{m_{n_j}^{(i)}}{1-\beta_1^i},\quad \hat{v}_{n_j}^{(i)}=\frac{v_{n_j}^{(i)}}{1-\beta_2^i}. \tag{35}$$

The multipliers are then updated as

$$\lambda_{n_j}^{(i)}\leftarrow\lambda_{n_j}^{(i-1)}+\eta\frac{\hat{m}_{n_j}^{(i)}}{\sqrt{\hat{v}_{n_j}^{(i)}}+\epsilon} \tag{36}$$

where $\epsilon$ is a small constant.

Since the constraint incorporated into the objective function is a less-than-or-equal-to inequality, a projection step is required to ensure that the corresponding multipliers remain nonnegative, and we have $\lambda_{n_j}^{(i)}\leftarrow\max(0,\hat{\lambda}_{n_j}^{(i)})$.

The pseudocode for the proposed algorithm to solve the long-term optimization problem is presented in Algorithm 1.

### C. Complexity Analysis for the Long-Term Optimization

The computational complexity mainly results from the greedy-strategy-based heuristic algorithm for inner-layer iteration. The key step is to solve the greedy-inspired knapsack problem to obtain the caching matrix, whose complexity is $\mathcal{O}(K(\log K))$ [34]. With the total number of iterations set as $T_{\text{iter}}$, the overall complexity is calculated as $\mathcal{O}(T_{\text{iter}}\cdot N\cdot K(\log K))$.

## V. Proposed Method for the Short-Term Optimization Problem

This section proceeds to solve the short-term problem formulated in Section III-B.

### A. Two-Stage Decoupled Optimization Framework

The joint optimization formulated in (21) is a highly complex mixed-integer nonlinear programming (MINLP) problem. Directly solving it within a continuous MARL action space is

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1** Iterative Algorithm for Solving Problem (19)

**Input:** $\mathcal{N}$; $\mathcal{K}$; $\{s_k\}_{k\in\mathcal{K}}$; $\{S_{n_j}\}_{n_j\in\mathcal{N}}$; historical requests; initial caching state $\mathbf{X}(t'-T_{\mathrm{L}})$; parameters $\alpha,\beta,\gamma_1,\gamma_2,T_{\mathrm{iter}},\eta$, and $\varepsilon$.

**Initialization:**
  Set $t=0$;
  Calculate $\{N_k\}_{k\in\mathcal{K}}$;
  Use the LDA model to predict $\{P_k\}_{k\in\mathcal{K}}$;
  Obtain the service request probability by (13) and calculate $\rho_{k,n_j}$;
  $\lambda_{n_j}^{(0)}\leftarrow 0$; $m_{n_j}^{(0)}\leftarrow 0$; $v_{n_j}^{(0)}\leftarrow 0$; $\mathbf{X}^*(t')\leftarrow\emptyset$, and $\mathbf{U}^*\leftarrow -\infty$.
**for** $i=1$ to $T_{\mathrm{iter}}$ **do**
  Obtain feasible matrix $\mathbf{X}(t')$ by solving (30);
  Update $\hat{\mathbf{X}}\leftarrow\mathbf{X}(t')$;
  **if** $\hat{\mathbf{X}}$ is feasible and $\mathbf{U}(\hat{\mathbf{X}})>\mathbf{U}^*$ **then**
    $\mathbf{X}^*(\mathbf{t}')\leftarrow\hat{\mathbf{X}}$ and $\mathbf{U}^*\leftarrow U(\hat{\mathbf{X}})$;
  **end if**
  **for each** UAV $n_j\in\mathcal{N}$ **do**
    Calculate the gradients by (32);
    Update the first-order and the second-order moments by (33) and (34);
    Compute the first-order and the second-order bias-corrected moments by (35);
    Update Lagrangian multipliers by (36);
    $\lambda_{n_j}^{(i)}\leftarrow\max(0,\lambda_{n_j}^{(i)})$;
    Calculates $\hat{\mathbf{U}}_{\mathbf{dual}}^i$ and the relative dual gap as $\Delta_{\mathrm{rel}}^{(i)}=\frac{\hat{\mathbf{U}}_{\mathbf{dual}}^i-\mathbf{U}^*}{\max|\mathbf{U}^*|}$
    **if** $\Delta_{\mathrm{rel}}^{(i)}\le\varepsilon$;
    **break**
    **end if**
  **end for**
**end for**
**Output:** Optimal service caching matrix $\mathbf{X}^*(\mathbf{t}')$.

intractable. Therefore, we decompose it into a decoupled two-stage sequential decision process at each time slot $t$, shown as follows.

*Stage 1 (Trajectory Optimization):* Agents explore the continuous spatial domain. Driven by minimizing system delay, agents utilize a self-attention mechanism to output continuous actions determining the optimal UAV trajectories $\mathbf{P}$.

*Stage 2 (Task Offloading and UAV-User Association):* Given $\mathbf{P}$, the discrete variables $\mathbf{A}$, $\mathbf{L}$, $\mathbf{E}$, and $\mathbf{C}$ are deterministically resolved via a delay-minimizing greedy strategy for each task $M_{u_i}$, given as

1) *Local:* If $M_{u_i}$ can be processed locally within $\Delta t$, set $l_{u_i}(t)=1$.
2) *Edge:* Otherwise, user $u_i$ associates with the reachable UAV $n_j^*$ that caches program $K_{u_i}$ and minimizes the total edge delay. We set $a_{n_j^*,u_i}(t)=e_{n_j^*,u_i}(t)=1$, where $n_j^*=\arg\min_{n_j\in\mathcal{N}}(T_{M_{u_i},n_j}^{ul}+T_{n_j,u_i}^{of})$.
3) *Cloud:* If no reachable UAV caches $K_{u_i}$, $u_i$ connects to the geographically nearest UAV $n_j^*$ to relay the task to the cloud, setting $a_{n_j^*,u_i}(t)=c_{n_j^*,u_i}(t)=1$.

### B. Short-Term Trajectory Optimization

With obtained service caching decisions, the aim of short-term optimization is to optimize the UAV flight trajectories and task offloading decisions. To deal with this challenging problem, the problem is transformed into a multiagent Markov decision process (MAMDP). We then employ the MAPPO algorithm to find its solution.

In the training process, we employ the CTDE paradigm [35]. The centralized critic leverages the global information to guide the learning processes for all agents. During the execution phase, each UAV acts as an independent agent. We model the short-term problem as an MAMDP, represented by the tuple $\mathcal{M}=(\mathcal{S},\mathcal{A},P,R,\gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P$ denotes the state transition probabilities, $R$ is the reward function, and $\gamma$ represents the discount factor. Key elements of the MDP are specified as follows.

*State Space ($\mathcal{S}$):* It is a high-dimensional continuous vector composed of the state observations from multiple UAVs. The state $\mathbf{s}(t)\in\mathcal{S}$ represents a complete observation of the environment at time slot $t$. The observation vector $\mathbf{obs}_{n_j}(t)$ for UAV $n_j$ includes UAV coordinates, normalized distances to four boundaries, user coordinates, normalized size of each requested program, and caching indicators. With $N$ edge nodes, the global state vector is defined as

$$\mathbf{s}(t)=[\mathbf{obs}_{n_1}(t),\mathbf{obs}_{n_2}(t),\ldots,\mathbf{obs}_{n_j}(t),\ldots,\mathbf{obs}_{n_N}(t)]. \tag{37}$$

*Action Space ($\mathcal{A}$):* The action space $\mathcal{A}$ is the Cartesian product of the individual action spaces of all agents, i.e., $\mathcal{A}=\mathcal{A}_{n_1}\times\mathcal{A}_{n_2}\times\cdots\times\mathcal{A}_{n_N}$. The joint action is denoted as $\mathbf{a}(t)=[\mathbf{a}_{n_1}(t),\mathbf{a}_{n_2}(t),\ldots,\mathbf{a}_{n_N}(t)]$. In contrast to the centralized approach, $\mathbf{a}_{n_j}(t)$ is determined by policy $\pi_{n_j}$ obtained from its independent actor network. The action vector of UAV $n_j$ is represented as $\mathbf{a}_{n_j}(t)=[\mathbf{a}_{n_j}^x(t),\mathbf{a}_{n_j}^y(t)]$, which gives the movement direction and the relative speed. The policy network uses an activation function to constrain each component of the action vector to the range of $[-1,1]$. Then, the action vector is converted into a physical displacement vector $\Delta\mathbf{p}_{n_j}(t)$, as given by

$$\Delta\mathbf{p}_{n_j}(t)=\left[\mathbf{a}_{n_j}^x(t)\cdot D_{\max},\mathbf{a}_{n_j}^y(t)\cdot D_{\max}\right]. \tag{38}$$

The next position is updated by

$$\mathbf{p}_{n_j}(t+1)=\mathbf{p}_{n_j}(t)+\Delta\mathbf{p}_{n_j}(t). \tag{39}$$

*State Transition Probability ($P$):* The state transition probability, represented by $P(\mathbf{s}(t+1)|\mathbf{s}(t),\mathbf{a}(t))$, describes the likelihood of moving to a new state after the agent performs an action. Since we employ a model-free RL approach, the state transition probability is not explicitly modeled. Instead, they are affected by the deterministic UAV movement.

*Reward Function ($R$):* The reward function $R(\mathbf{s}(t),\mathbf{a}(t))$ is designed to evaluate the quality of action $\mathbf{a}(t)$ with given $\mathbf{s}(t)$. The immediate reward at time slot $t$, denoted as $r_t$, is constructed as a weighted combination of several components, as given by

$$r_t=\omega_1 r_t^{\mathrm{de}}+\omega_2 r_t^{\mathrm{co}}-\omega_3 r_t^{\mathrm{ds}}-\omega_4 r_t^{\mathrm{bd}}-\omega_5 r_t^{\mathrm{ns}} \tag{40}$$

where $w_1,w_2,w_3,w_4$, and $w_5$ are weighting coefficients. In (40), $r_t^{\mathrm{de}}$ is the delay improvement reward, and it rewards the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

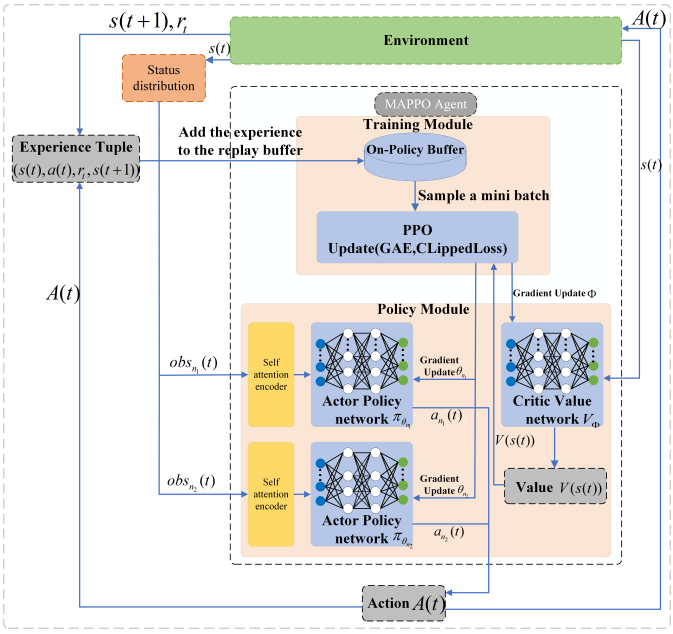



Fig. 3. Explanation of the MAPPO algorithm based on the self-attention mechanism.

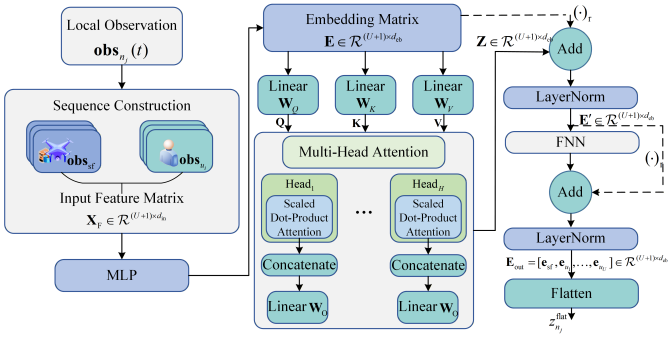



Fig. 4. Encoder observation of each UAV based on the self-attention mechanism.

delay improvement relative to the baseline where all tasks are offloaded to the cloud center. Service count reward, denoted by $r_t^{\text{co}}$, is a fixed positive reward and given for each task if it is successfully processed by a UAV, explicitly encouraging each agent to maximize the number of its serving users. Average service distance penalty $r_t^{\text{ds}}$ is a nonlinear term, obtained by considering the average distance between each UAV and its serving users. It incentivizes the agent not only to serve users but also to shorten the geographical distances between them. Boundary penalty $r_t^{\text{bd}}$ is applied when a UAV flies near the area boundaries. No-serve penalty $r_t^{\text{ns}}$ will appear if the tasks are offloaded but not conducted by UAVs. The weighting coefficients primarily balance the magnitudes of various factors during the training process to prevent excessive differences that may cause training failure or prevent convergence. We have demonstrated the validity of the weighting values through magnitude analysis and a series of sensitivity experiments.

*Discount Factor ($\gamma$):* $\gamma \in [0, 1]$ is used to balance the importance between immediate and future rewards. For the agent, the objective of the short-term optimization is to find the optimal policy $\boldsymbol{\pi}^* = [\pi_{n_1}^*, \pi_{n_2}^*, \ldots, \pi_{n_N}^*]$ that maximizes the expected value of the cumulative discounted reward from any initial state, satisfying

$$\boldsymbol{\pi}^* = \arg\max_{\boldsymbol{\pi}} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(\mathbf{s}(t), \mathbf{a}(t)) | \boldsymbol{\pi}\right]. \tag{41}$$

Based on the actor–critic architecture, we employ the MAPPO algorithm, and follow the CTDE paradigm to solve the short-term MAMDP problem, as shown in Fig. 3. Compared to traditional single-agent algorithms, MAPPO is more efficient and stable when handling multi-UAV collaborative exploration tasks and working in continuous action spaces. During the training process, the MAPPO utilizes a clipped surrogate objective mechanism. This allows us to use a batch of sampled data for parameter updating, and excessive policy updates are prevented.

As the key of the MAPPO algorithm, the actor–critic architecture comprises two types of neural networks with distinct functions, illustrated as follows.

1) *Decentralized Actor Policy Networks:* Parameterized by $\theta_{n_j}$, agent $n_j$ possesses an independent actor network. This network is responsible for learning the optimal policy $\pi_{\boldsymbol{\theta}_{n_j}}^*(\mathbf{a}(t)|\mathbf{s}(t))$ to maximize the cumulative reward. Based on local observations, it generates a flight action.
2) *Centralized Critic Value Network:* Parameterized by $\boldsymbol{\phi}$, only one centralized critic network is included. It evaluates the quality of the overall policy. The input is the global state $\mathbf{s}(t)$, and the output is the state-value function $V(\mathbf{s}(t))$. $V(\mathbf{s}(t))$ can help predict the cumulative reward that can be obtained by obeying the policy.

### C. Self-Attention Encoder

In the considered multiagent system, the decision-making quality of each UAV highly depends on its understanding of its own state and its complex relationships with all users. Traditional MLP struggles to effectively capture the dynamic relationships between the UAV and users when processing complex data. As shown in Fig. 4, we integrate an SAE at the front-end of the policy network for each agent. The reasons for adopting the SAE in the MAPPO model are illustrated in the following. First, in real-world postdisaster scenarios, user tasks are generated with a high degree of intermittency. In each time slot, only a subset of users may generate task processing requests, and a significant amount of zero-padding will appear in the observation vectors. By embedding the SAE into the front end of the MAPPO model, it can act as a dynamic noise filter to assign near-zero attention weights to inactive users. This integration directly provides the actor network of the MAPPO model with clean and stable feature representations, significantly accelerating the convergence speed and performance stability. Second, postdisaster environments are typically characterized by high mobility and topological fluctuations. When a CTDE architecture is employed, each UAV must make decisions independently during the execution phase, relying solely on its local actor network. Unlike traditional MLPs, which strictly bind the inputs to their indices, the SAE inherently possesses the characteristic of permutation

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

invariance. Combining MAPPO and SAE ensures that the decisions made by the distributed actor network do not depend on fixed indices, and user priorities can be evaluated based on relative contextual features. This means that, even when user distributions or scales undergo drastic changes, each agent in the MAPPO model can still maintain a higher degree of decision-making robustness. Finally, for the joint optimization in multi-UAV scenarios, it is challenging to avoid resource conflicts. To solve this issue, the SAE is used to combine physical space with service caching states, enabling collaborations among multiple UAVs. Guided by the rewards from the centralized critic network, the distributed actor network equipped with an SAE can intelligently distribute user tasks according to the attention scores by considering geographical locations and caching efficiency. The UAV prioritizes flying toward the user clusters that the UAV can serve with its local cached programs, which decreases the task processing latency without incurring additional communication overheads.

The input to the SAE is an observation sequence, containing information about the current UAV and all users. Through self-attention calculation, a flattened feature vector encapsulating the user's importance is output. The self-attention calculation process is described as follows.

For agent $n_j$, its local observation $\mathbf{obs}_{n_j}(t)$ is analyzed to construct a sequence. This sequence comprises the UAV's own feature vector, defined as $\mathbf{obs}_{\mathrm{sf}}$, and the feature vector of user $u_i$, denoted by $\mathbf{obs}_{u_i}$. We regard the sequence as the input feature matrix $\mathbf{X}_{\mathrm{F}} \in \mathcal{R}^{(U+1)\times d_{\mathrm{in}}}$, where $U+1$ is the length of the input sequence and $d_{\mathrm{in}}$ is the dimension of the raw feature vectors.

The input feature matrix $\mathbf{X}_{\mathrm{F}}$ is passed through the linear layers of an MLP structure and projected into a high-dimensional embedding space to obtain the embedding matrix $\mathbf{E} \in \mathcal{R}^{(U+1)\times d_{\mathrm{eb}}}$, shown as follows:

$$\mathbf{E} = \mathrm{MLP}(\mathbf{X}_{\mathrm{F}}) \tag{42}$$

where $d_{\mathrm{eb}}$ is the embedding dimension.

To enable the model to simultaneously acquire the information from different positions of the input sequence within various representation sub-spaces, we employ a multihead attention mechanism in the SAE. The embedding matrix $\mathbf{E}$ is passed in parallel through three independent linear transformation layers, and it is multiplied by the weight matrices $\mathbf{W}_Q, \mathbf{W}_K$, and $\mathbf{W}_V$ to generate the query, key, and value matrices, which are shown as

$$\mathbf{Q} = \mathbf{E}\cdot\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{E}\cdot\mathbf{W}_K, \ \mathbf{V} = \mathbf{E}\cdot\mathbf{W}_V. \tag{43}$$

To enable the agent to comprehensively learn the dependencies within the input sequences, we split $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ along with their feature dimension into $H$ independent heads. For the $h$th head, the respective query, key, and value matrices are denoted as $\mathbf{Q}_h$, $\mathbf{K}_h$, and $\mathbf{V}_h \in \mathcal{R}^{(U+1)\times d_k}$, respectively, where $d_k = d_{\mathrm{eb}}/H$. The scaled dot-product attention is then computed as

$$\begin{aligned}\mathrm{head}_i &= \mathrm{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h)\\ &= \mathrm{softmax}\left(\frac{\mathbf{Q}_h\mathbf{K}_h^T}{\sqrt{d_k}}\right)\mathbf{V}_h.\end{aligned} \tag{44}$$

The outputs of all heads are concatenated along the feature dimension, and then fused through a final linear projection layer with the weight matrix $\mathbf{W}_{\mathrm{O}} \in \mathcal{R}^{d_{\mathrm{eb}}\times d_{\mathrm{eb}}}$. The final multihead attention output matrix $\mathbf{Z} \in \mathcal{R}^{(U+1)\times d_{\mathrm{eb}}}$ is yielded as

$$\begin{aligned}\mathbf{Z} &= \mathrm{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V})\\ &= \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\mathbf{W}_{\mathrm{O}}.\end{aligned} \tag{45}$$

We define the residual connection operation as $(\cdot)_{\mathrm{r}}$, and then apply residual connections to $\mathbf{Z}$ and $\mathbf{E}$, followed by the layer normalization for stabilization, as given by

$$\mathbf{E}' = \mathrm{LayerNorm}((\mathbf{E})_{\mathrm{r}} + (\mathbf{Z})_{\mathrm{r}}) \tag{46}$$

where $\mathbf{E}'$ is then fed into a feed-forward network (FFN), where another residual connection and layer normalization are applied to obtain the following output of the SAE:

$$\mathbf{E}_{\mathrm{out}} = \mathrm{LayerNorm}((\mathbf{E}')_{\mathrm{r}} + \mathrm{FFN}((\mathbf{E}')_{\mathrm{r}})). \tag{47}$$

Containing rich contextual information about current UAV and users, $\mathbf{E}_{\mathrm{out}}$ is a feature sequence vector, which is represented as $[\mathbf{e}_{\mathrm{sf}}, \mathbf{e}_{u_1}, \ldots, \mathbf{e}_{u_U}]$.

The first feature vector from the output sequence $\mathbf{E}_{\mathrm{out}}$ corresponds to the UAV's self-feature $\mathbf{e}_{\mathrm{sf}}$, which has been updated by the attention mechanism to aggregate global contextual information from all users, and we attain

$$\mathbf{z}_{n_j}^{\mathrm{ct}} = \mathbf{E}_{\mathrm{out}}[0]. \tag{48}$$

### D. MAPPO Algorithm

In the CTDE framework, the decentralized actor networks are responsible for making independent decisions. The centralized critic network evaluates the overall quality of the decisions and provides feedback to guide parameter updates for all actor networks.

*1) Update of the Critic Network:* The update goal is to make the state-value function $V(\mathbf{s}(t))$ as close as possible to the calculated actual return, which is given by

$$V_t(\mathrm{tg}) = r_t + \gamma\cdot V(\mathbf{s}(t+1)). \tag{49}$$

The mean squared error (mse) is used as the loss function, calculated as

$$L^{\mathrm{C}}(\boldsymbol{\phi}) = [V(\mathbf{s}(t)) - V_t(\mathrm{tg})]^2. \tag{50}$$

After obtaining the gradient of the loss function with respect to $\boldsymbol{\phi}$, $\boldsymbol{\phi}_t$ is updated by

$$\boldsymbol{\phi}_t = \boldsymbol{\phi}_{t-1} - \alpha_c\nabla_{\boldsymbol{\phi}}L^{\mathrm{C}}(\boldsymbol{\phi}) \tag{51}$$

where $\alpha_c$ is the learning rate for the critic network.

*2) Update of the Actor Network:* To update the actor network, we construct the loss function as

$$L^{\mathrm{A}}(\boldsymbol{\theta}) = -L^{\mathrm{CP}}(\boldsymbol{\theta}) - c_{\mathrm{en}}\cdot\mathbb{E}_t[S(\pi_{\boldsymbol{\theta}}|\mathbf{z}_{\mathrm{gb}}(t))] \tag{52}$$

where $c_{\mathrm{en}}$ is the entropy coefficient for controlling the exploration strength during the training phase, and $\boldsymbol{\theta} = [\boldsymbol{\theta}_{n_1}, \boldsymbol{\theta}_{n_2}, \ldots, \boldsymbol{\theta}_{n_N}]$. $\mathbf{z}_{\mathrm{gb}}(t) = [\mathbf{z}_{n_1}^{\mathrm{ct}}(t), \mathbf{z}_{n_2}^{\mathrm{ct}}(t), \ldots, \mathbf{z}_{n_N}^{\mathrm{ct}}(t)]$ represents

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

the global context feature. $L^{\mathrm{CP}}(\boldsymbol{\theta})$ is the MAPPO surrogate objective function, as given by

$$L^{\mathrm{CP}}(\boldsymbol{\theta}) = \mathbb{E}_t\Big[\min\left(r_I(\boldsymbol{\theta})\hat{A}_t^{\mathrm{norm}}, \mathrm{clip}(r_I(\boldsymbol{\theta}), 1-\epsilon, 1+\epsilon)\hat{A}_t^{\mathrm{norm}}\right)\Big] \tag{53}$$

where $\epsilon$ is the hyperparameter to show the clipping range. The value of $r_I(\boldsymbol{\theta})$ is constrained to the interval of $[1-\epsilon, 1+\epsilon]$.

For the actor network of the $j$th agent, $\mathbf{obs}_{n_j}(t)$ is first provided as the input. This observation vector is initially processed by the SAE to generate a context-aware feature vector $\mathbf{z}_{n_j}^{\mathrm{ct}}(t)$. This vector is then passed through MLP layers, and we ultimately yield a mean value and a log standard deviation of $\theta_{n_j}$, denoted by $\mu_{\theta_{n_j}}$ and $\log\sigma_{\theta_{n_j}}$, respectively. The action distribution for the new policy of the $j$th actor network is established as

$$\pi_{\boldsymbol{\theta}_{n_j}}\left(\cdot\middle|\mathbf{z}_{n_j}^{\mathrm{ct}}(t)\right) = \mathcal{N}\left(\mu_{\theta_{n_j}}, \exp(\log\sigma_{\theta_{n_j}})\right). \tag{54}$$

To correct the distribution bias between the old and new strategies, the importance ratio is supposed to be calculated. For numerical stability, we choose to obtain the ratio in the following logarithmic domain:

$$r_I(\boldsymbol{\theta}) = \exp\left(\log\pi_{\boldsymbol{\theta}}(\mathbf{a}(t)|\mathbf{z}_{\mathrm{gb}}(t)) - \log\pi_{\boldsymbol{\theta}_{\mathrm{old}}}(\mathbf{a}(t)|\mathbf{z}_{\mathrm{gb}}(t))\right). \tag{55}$$

Due to the lack of a joint policy network, we use the new policy distribution $\pi_{\boldsymbol{\theta}_{n_j}}$ to reevaluate the log-probability of the individual action $\mathbf{a}_{n_j}(t)$, i.e., $\log\pi_{\boldsymbol{\theta}_{n_j}}(\mathbf{a}_{n_j}(t)|\mathbf{z}_{n_j}^{\mathrm{ct}}(t))$. The log-probability of the joint action is then transformed into

$$\log\pi_{\boldsymbol{\theta}}(\mathbf{a}(t)|\mathbf{z}_{\mathrm{gb}}(t)) = \sum_{n_j=1}^{N}\log\pi_{\boldsymbol{\theta}_{n_j}}\left(\mathbf{a}_{n_j}(t)\middle|\mathbf{z}_{n_j}^{\mathrm{ct}}(t)\right). \tag{56}$$

The advantage function $A(\mathbf{s}(t), \mathbf{a}(t)) = Q(\mathbf{s}(t), \mathbf{a}(t)) - V(\mathbf{s}(t))$ is used to measure the superiority of the joint action $\mathbf{a}(t)$ performed in state $\mathbf{s}(t)$, where $Q(\mathbf{s}(t), \mathbf{a}(t)) = \mathbb{E}[r_t + \gamma V_{\boldsymbol{\phi}}(\mathbf{s}(t+1)) \mid \mathbf{s}(t), \mathbf{a}(t)]$. In order to obtain more stable and accurate estimates, we use generalized advantage estimation (GAE) to balance the tradeoff between bias and variance. First, $V(\mathbf{s}(t))$ is used to calculate the single-step temporal difference (TD) error, which serves as an initial estimate for the advantage function. The TD error is provided as

$$\delta_t = r_t + \gamma V(\mathbf{s}(t+1)) - V(\mathbf{s}(t)). \tag{57}$$

Although the single-step TD error is a low-variance estimate, it is highly susceptible to bias. To deal with the issue, a hyperparameter $\varsigma \in [0, 1]$ is introduced to compute an exponentially weighted average of multistep TD errors, which is shown as

$$\hat{A}_t^{\mathrm{GAE}}(\gamma, \varsigma) = \sum_{l=0}^{\infty}(\gamma\varsigma)^l\delta_{t+l}. \tag{58}$$

To further stabilize the training process, the calculated GAE advantage values are normalized, having a mean of 0 and a variance of 1, and we obtain that

$$\hat{A}_t^{\mathrm{norm}} = \frac{\hat{A}_t^{\mathrm{GAE}} - \mathrm{mean}(\hat{A})}{\mathrm{std}(\hat{A}) + \epsilon} \tag{59}$$

where $\hat{A}$ represents the entire batch of calculated advantage values. To this end, by substituting (59) into (53), $L^{\mathrm{CP}}(\boldsymbol{\theta})$ is obtained.

To prevent the policy from converging to a suboptimal solution, an entropy reward term in (52) is introduced, shown as

$$S(\pi_\theta|\mathbf{z}_{\mathrm{gb}}(t)) = \sum_{n_j=1}^{N} H\left(\pi_{\boldsymbol{\theta}_{n_j}}\left(\cdot\middle|\mathbf{z}_{n_j}^{\mathrm{ct}}(t)\right)\right) \tag{60}$$

where $H$ denotes the Shannon entropy.

From the above, by plugging (53) and (60) into (52), the loss function for the actor network is achieved. The final objective is to minimize the loss function $L^{\mathrm{A}}(\boldsymbol{\theta})$. Then, $\boldsymbol{\theta}$ is updated as

$$\boldsymbol{\theta} = \boldsymbol{\theta}_{\mathrm{old}} - \alpha_a\nabla_{\boldsymbol{\theta}}L^{\mathrm{A}}(\boldsymbol{\theta}) \tag{61}$$

where $\alpha_a$ is the learning rate for the actor network.

### *E. Complexity Analysis for the Short-Term Optimization*

The complexity of the short-term algorithm is primarily determined by the number of agents, the number of users, and the neural network architecture. Moreover, we have omitted the complexity $\mathcal{O}(UN)$ caused by the greedy strategy. $L = 1+U$ denotes the length of the attention sequence; $T$ is the number of interactions between agents and the environment; $E$ is the number of required iterations per update; $c_{\mathrm{bp}}^{(A)}$ and $c_{\mathrm{bp}}^{(C)}$ are backpropagation amplification coefficients; $d_{\mathrm{eb}}$, $H_A$, $D_{\mathrm{lo}}$, and $D_{\mathrm{gl}}$ represent the dimensions of attention embedding, two-layer MLP backend in the actor network, local observation of each agent, and the global state; and $H_C$ gives the hidden dimension of the two-layer MLP in the critic network.

*Actor Network:* Each actor network incorporates an SAE to focus on its correlations with users. Consequently, the forward propagation complexity comprises the complexity of the SAE and two MLP layers. The complexity caused by backpropagation is typically related to that of the forward propagation [36]. Therefore, the computational complexity resulting from the forward and backward propagations is shown as

$$F_{\mathrm{Actor}}^{\mathrm{fp}} = \mathcal{O}\left(\underbrace{L^2d}_{\text{correlation}} + \underbrace{Ld^2}_{\substack{\text{linear projection}\\ \&\text{FFN}}} + \underbrace{LdH_A + H_A^2}_{\text{MLP}}\right) \tag{62}$$

$$F_{\mathrm{Actor}}^{\mathrm{bp}} = \mathcal{O}\left(c_{\mathrm{bp}}^{(A)}\cdot F_{\mathrm{Actor}}^{\mathrm{fp}}\right). \tag{63}$$

*Critic Network:* The centralized critic network uses an MLP architecture to process the global state. The forward propagation complexity only comes from the MLP layer, while the backward propagation complexity is also related to that of the forward propagation, which is shown as

$$F_{\mathrm{Critic}}^{\mathrm{fp}} = \mathcal{O}\left(D_{\mathrm{gl}}H_C + H_C^2\right) \tag{64}$$

$$F_{\mathrm{Critic}}^{\mathrm{bp}} = \mathcal{O}\left(c_{\mathrm{bp}}^{(C)}\cdot F_{\mathrm{Critic}}^{\mathrm{fp}}\right). \tag{65}$$

Next, we analyze the complexity generated by different phases.

*1) Data Collection Phase:* During this phase, each agent interacts with the environment for $T$ times. Each interaction requires performing forward propagations for both the actor network and the critic network. Therefore, the resulting computational complexity is given as $\mathcal{O}(T[NF_{\mathrm{Actor}}^{\mathrm{fp}} + F_{\mathrm{Critic}}^{\mathrm{fp}}])$.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

*2) Training Phase:* This stage requires traversing the tuples obtained from $T$ times of interactions during the data collection phase for $E$ times. Each iteration necessitates performing one forward pass and one backpropagation pass for each actor and critic network. The complexity is given by $\mathcal{O}(ET[N(F^{\text{fp}}_{\text{Actor}} + F^{\text{bp}}_{\text{Actor}}) + (F^{\text{fp}}_{\text{Critic}} + F^{\text{bp}}_{\text{Critic}})])$.

Therefore, the overall complexity comes from the two phases and is written as $\mathcal{O}(ET[N(F^{\text{fp}}_{\text{Actor}} + F^{\text{bp}}_{\text{Actor}})])$. After further simplifications, the final complexity is obtained as $\mathcal{O}(L^2d + Ld^2 + LdH_A)$.

Although the self-attention mechanism exhibits quadratic complexity with respect to the number of users, it is able to process the input sequences with changing length, while the traditional MLP cannot achieve this goal. In dynamic UAV-MEC networks, the characteristics of served users frequently change. The attention mechanism enables trained policies to adapt to varying user scales without retraining, finally achieving architectural scalability. Therefore, the attention mechanism with quadratic computational cost can improve the network flexibility. When the number of users increases slightly, the increase in complexity is affordable for existing central processing units (CPUs). When there are massive numbers of users to be included in the network, more computing resources are needed, which can be satisfied by resorting to the central cloud center for remote processing. The time required for training the model to process environmental states and generate action outputs is defined as the inference time. Even in a large-scale edge computing scenario with 500 users, the single-step inference time is only 2.4 ms. Using parallel computing can further reduce this time, demonstrating the validity and scalability of the proposed algorithm for practical deployment.

## VI. Simulation Results

In this section, we show the simulation results of our long-term and short-term designs. All simulation experiments are conducted on a workstation configured with an Intel Core i9 processor, 32-GB RAM, and a GeForce RTX 4060 graphics card from NVIDIA. The algorithms are implemented in Python 3.11.8, and the neural networks are built and trained using PyTorch 2.10.0 with a deep learning framework.

### A. Long-Term Simulations

Table I presents the primary parameters used in the long-term simulations. Through sensitivity analysis experiments, the cache hit rate achieves the peak performance when $\gamma_1$ is set within the range of [0.6, 0.7]. We select $\gamma_1 = 0.65$, at which point the model achieves the near-optimal performance. To evaluate the system performance of the proposed method, we introduce three baseline algorithms for comparison: least frequently used (LFU) [37], least recently used (LRU) [38], and greedy stochastic priorit (GSP) strategies [39].

We treat the relative dual gap as a performance boundary for measuring the near-optimality of the solution obtained by our proposed algorithm. Under different initial learning rates, Fig. 5 illustrates the convergence performance of the relative duality gap, and we find that the proposed algorithm converges within a 1% relative gap. The figure also shows that an appropriate initial learning rate can accelerate the convergence speed.

Fig. 6 compares the CHR of the proposed algorithm with three baselines under different caching capacities. We define the cache hit rate as the ratio of the number of service requests processed locally by the server to the total number of user

TABLE I
Simulation Parameters of Long-Term Optimization

| Parameters | Values |
|---|---|
| Number of users ($U$) | 8 |
| Number of UAVs/Servers ($N$) | 2 |
| Number of services ($K$) | 300 |
| Number of topics ($R$) | 5 |
| Size of program $k$ ($s_k$) | {3, 5} Mbits |
| Storage cost weight ($\alpha$) | 0.01 |
| Replacement cost weight ($\beta$) | 0.05 |
| Popularity prediction weight ($\gamma_1$) | 0.65 |
| Historical count weight ($\gamma_2$) | 0.35 |
| Total number of iterations | 20 |
| Initial learning rate | 0.001 |
| Maximal learning rate | 0.01 |
| Number of warm-up iterations | 2 |
| Adam optimizer parameters $\beta_1$ and $\beta_2$ | 0.75, 0.95 |

TABLE II
Simulation Parameters of Short-Term Optimization

| Parameter | Value |
|---|---|
| Max steps per episode | 60 |
| Task change interval | 10 time slots |
| Range of UAV communications | 150 m |
| Max UAV speed | 300 m/min |
| Channel bandwidth | 20 MHz |
| Transmit power of each user | 20 dBm |
| Path loss exponent | 3.5 |
| Number of training episodes | 2000 |
| Actor/Critic learning rate | $1 \times 10^{-4}$ |
| Discount factor ($\gamma$) | 0.99 |
| GAE lambda ($\lambda$) | 0.95 |
| Clip epsilon ($\epsilon$) | 0.2 |
| MAPPO epochs per update | 4 |
| Batch size for MAPPO | 256 |
| Entropy coefficient | 0.01 |
| Hidden layers | 2 layers |
| Attention embedding dimension | 128 |
| Environment parameters $a$ and $b$ | 4.88, 0.43 |

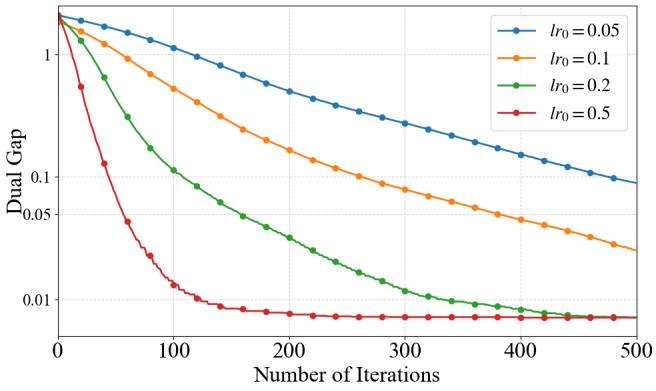



Fig. 5. Performance of relative dual gap.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

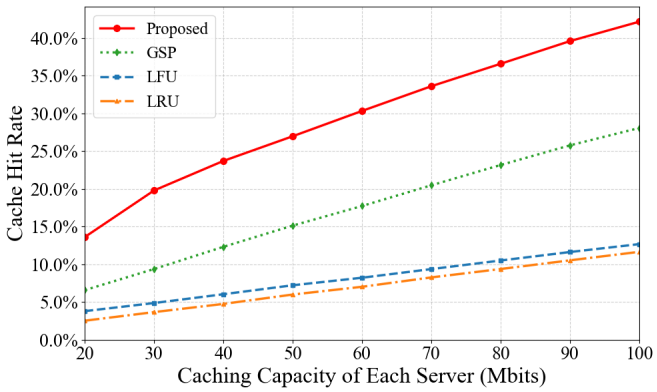



Fig. 6. Performance of CHR under different caching capacities.

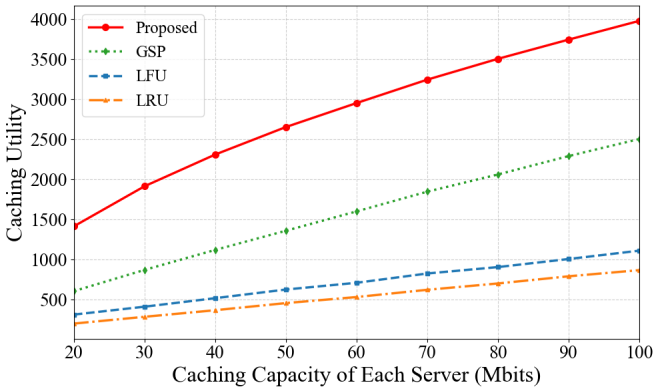



Fig. 7. Performance of the caching utility under different caching capacities of each server.

requests. The specific formula is shown as

$$\text{CHR} = \frac{\sum_{j\in\mathcal{J}} c_{j,n_j}}{J} \tag{66}$$

where $J$ represents the total number of user requests during the evaluation period, $j \in \mathcal{J}$ denotes the $j$th user request, and $c_{j,n_j}$ indicates whether the service program corresponding to the $j$th user request is cached by edge node $n_j$. Our proposed algorithm consistently gives the highest CHR, and the performance gain is 76.5%, as compared to the suboptimal baseline. Although the LFU and LRU are easy to implement, the greedy strategy used leads to almost identical cached contents across all servers. Compared to LFU, LRU is unable to retain long-term popular services. The GSP algorithm is fundamentally a local search approach, and cannot achieve a globally optimal configuration.

Fig. 7 compares the caching utility under different caching capacities. The proposed algorithm consistently outperforms all baselines, achieving a performance gain of 66% on average compared to the second-best method. As the caching capacity of edge servers increases, the caching utility is significantly enhanced. This is because, under constrained storage capacity, the system is forced to discard potentially useful contents. As the capacity increases, edge servers are able to store more service programs that align with popularity trends.

To comprehensively evaluate the impact of the popularity prediction weight $\gamma_1$ on the system performance, more

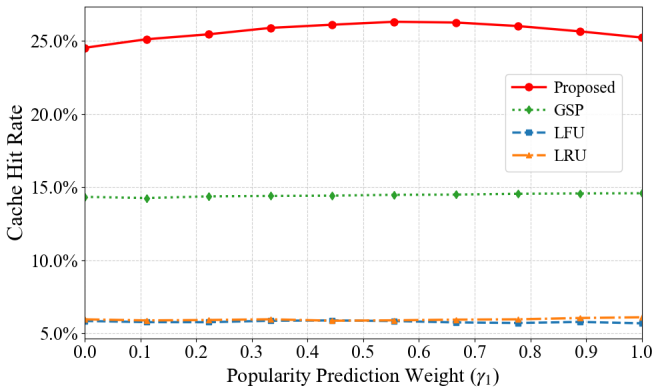



Fig. 8. Sensitivity analysis of the popularity prediction weight.

simulation experiments for sensitivity analysis are conducted. Fig. 8 illustrates the varying CHR under different values of $\gamma_1$. It can be observed that, as $\gamma_1$ rises, the CHR initially shows an upward trend. This is because assigning a higher weight to user preferences for the LDA model helps edge servers to more effectively capture upcoming service demands. However, when $\gamma_1$ exceeds 0.7, the CHR begins to decline. An excessively high $\gamma_1$ implies that the historical request trends, evaluated by $\gamma_2$, are almost ignored, and more prediction errors will appear. Therefore, when $\gamma_1$ is set within the range of [0.6, 0.7], the cache hit rate can achieve the optimum.

### B. Short-Term Simulations

Table II lists the main parameter values used in the short-term optimization. We set the five coefficients corresponding to the five components, i.e., $\omega_1$–$\omega_5$, as 0.1, 25.0, 0.1, 300.0, and 100.0, respectively. In an RL-based design, these coefficients are primarily used to serve as scaling factors for magnitude alignments, rather than representing the importance of each term. If uniform weights are used, high-magnitude terms would completely dominate the policy gradient, leading to reward sparsity in secondary tasks, such as boundary avoidance. The environment with $a = 4.88$ and $b = 0.43$ used in our communication model specifically represents a dense urban scenario, which is employed to simulate the postdisaster scenario with collapsed buildings [28]. To evaluate the performance of our proposed method, four baselines are used.

1) *Successive Convex Approximation (SCA)-Based Algorithm:* This is a common iterative algorithm by performing approximations to the nonconvex part, thereby transforming the original problem to be convex [40].
2) *Particle Swarm Optimization (PSO):* A myopic and greedy approach is adopted to find the optimal UAV positions [41].
3) *PPO:* This scheme is a single-agent version of our proposed method, conducted in a fully centralized manner [42].
4) *Random Trajectory (RT):* The UAV trajectories are generated randomly. At each time slot, each UAV will move a certain distance in a randomly generated direction.

For the baseline scheme of PPO, to ensure a fair comparison, its core RL hyperparameters are set to match those

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

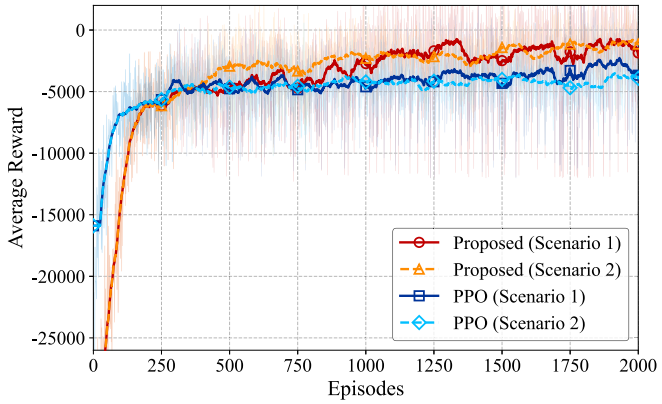



Fig. 9. Convergence behaviors in both scenarios.

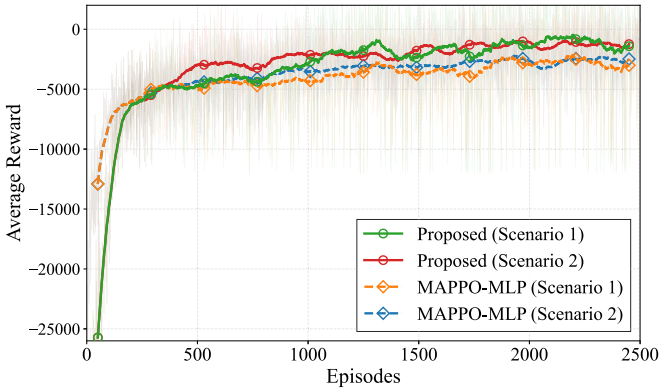



Fig. 10. Convergence performance of the proposed algorithm and ablation experiment in both scenarios.

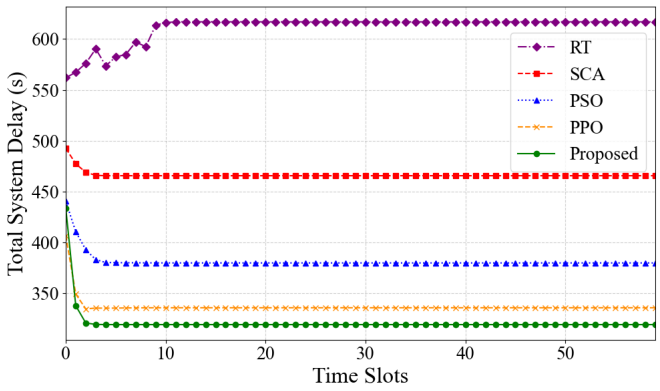



Fig. 11. Performance of the system processing delay from different algorithms.

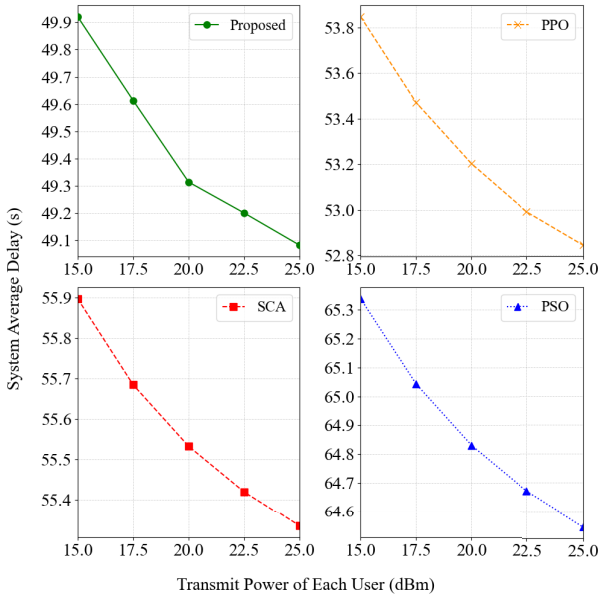



Fig. 12. Performance of the average delay versus the transmit power of each user.

presented in Table II. In addition, we have increased the number of hidden layers in the MLP to ensure that the total number of used parameters matches that of the proposed algorithm. For the baseline of PSO, the size of the particle swarm is set to 50, and the maximum number of iterations is 100. The inertia weight is linearly decreased from 0.9 to 0.4, and the cognitive and social coefficients, denoted by $c1$ and $c2$, are both set to 2.0. For the SCA-based benchmark, we set the maximum number of iterations to 100 and the convergence threshold to $10^{-4}$.

We divide the short-term simulation into two scenarios based on whether the task requests dynamically change. In a dynamic scenario, the SAE reassigns importance weights to ground users based on newly input observation vectors. Fig. 9 shows the convergence performance of the proposed MAPPO-attention algorithm and the centralized PPO in both scenarios. We find that the PPO experiences policy fluctuations and reward decay after the initial phase, indicating its difficulty in handling dynamic environments. In contrast, our proposed algorithm outperforms the baseline in terms of stability. This is because the centralized critic can provide global guidance, and the self-attention mechanism can capture key collaborative features. Hence, the proposed algorithm avoids the local optimum and results in a stable performance.

To verify the effectiveness and necessity of adopting the SAE within the proposed algorithm, Fig. 10 presents the varying average reward during the training process for the proposed algorithm and the MAPPO-MLP algorithm. The experimental results demonstrate that the proposed algorithm exhibits superior performance in both scenarios and indicate that the traditional MLP struggles to extract valuable user features under complex network topologies. To ensure a fair comparison, the number of neurons in the hidden layers of the MAPPO-MLP is set to 637, and the number of used parameters is 464 743, which is at a similar level to the proposed algorithm. From Fig. 10, we can conclude that the performance improvement does not stem from a simple increase in the number of used parameters, but from the structural advantages of the multihead self-attention mechanism in capturing the dynamic spatial coupling between UAVs and ground users.

*1) Scenario 1: (Environment With Static User Requests):* Fig. 11 reveals the system processing delay from different algorithms. The performance of all algorithms converges to stable levels after a rapid adjustment in a few time slots. As compared to PPO, the proposed algorithm further reduces the system delay by approximately 11.3%. The performance

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

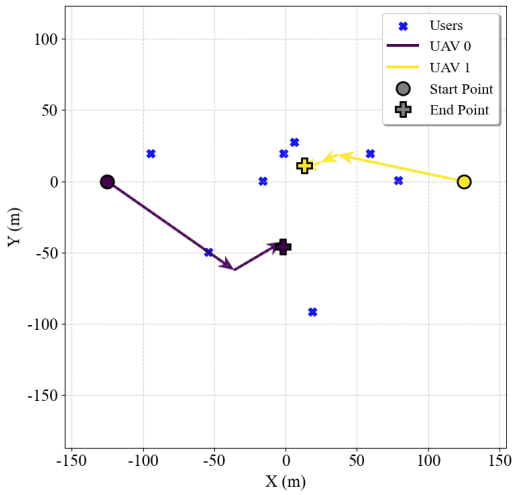



Fig. 13. UAV flight trajectory in scenario 1.

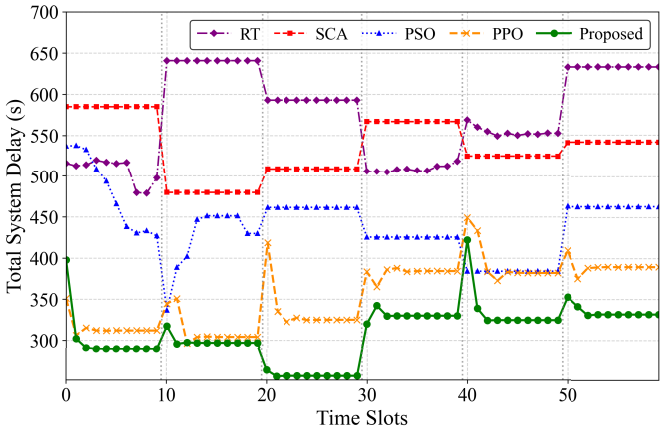



Fig. 14. Performance of total system processing delay in scenario 2.

improvement stems from its distributed actor networks. Meanwhile, the use of a centralized critic enables efficient task offloading and trajectory planning.

Fig. 12 shows the average latency per user under various transmit powers of the user. The proposed algorithm maintains the lowest delay. As the transmit power grows, the system delay of the proposed algorithm shows a slight downward trend. This is because a higher transmit power leads to better channel conditions, which in turn boost the data transmission rate and reduce the transmission delay. Compared to PPO, it reduces the system delay by approximately 7%.

Fig. 13 displays the UAV flight trajectories. Both UAV 0 and UAV 1 fly directly from their starting points to the optimal service centers to serve their target users. This type of efficient and proactive movement greatly reduces flight energy consumption. The reasons for the aforementioned phenomenon are specified as follows. First, the CTDE architecture enables each actor network to learn a refined policy in a low-dimensional local action space. Second, the self-attention mechanism allows each UAV to accurately estimate the position and importance of all users, thereby precisely calculating the optimal service center.

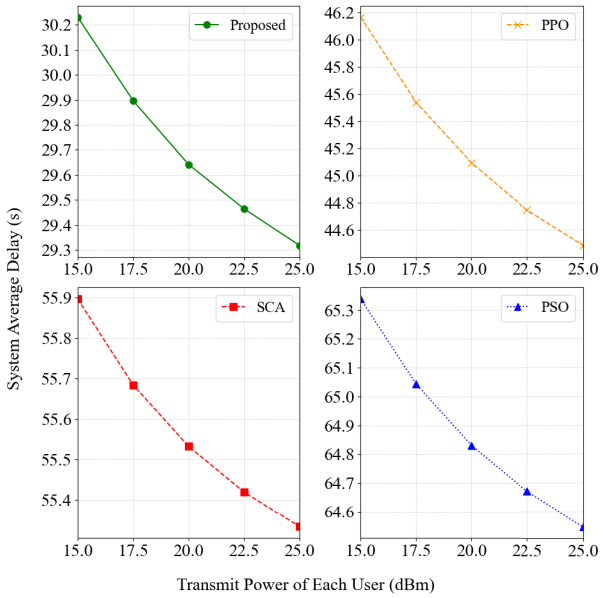



Fig. 15. Performance of the system average delay versus the transmit power of each user in scenario 2.

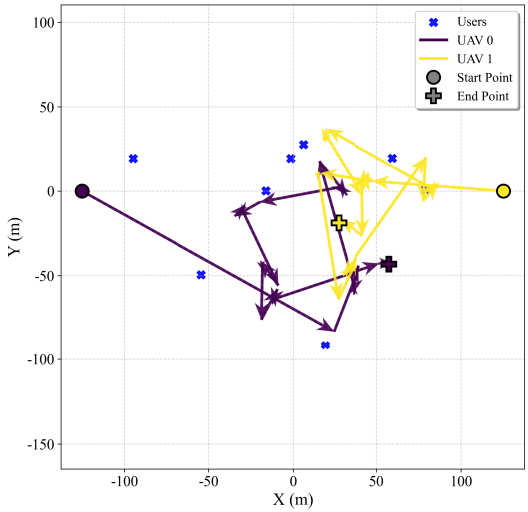



Fig. 16. UAV flight trajectory in scenario 2.

*2) Scenario 2: (Environment With Dynamic User Requests):* Fig. 14 reveals the convergence performance in the dynamic scenario. At the time slots of 9, 19, 29, 39, 49, and 59, the user request changes, and the delay immediately jumps. In this case, the proposed method can quickly find the optimal UAV positions, and a lower system processing delay is obtained. The proposed algorithm further reduces the system average delay by approximately 16% compared to the PPO. When new requests occur, the PPO algorithm is less sensitive and takes a longer time to stabilize. The delays for the SCA- and PSO-based algorithms show a stepwise pattern. This is because these two algorithms need complex iterations to find the optimal solution once the user requests a change.

Fig. 15 reveals how the delay varies with the transmit power of each user in the dynamic environment. The proposed algorithm demonstrates a significant performance gain over

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

the baselines, reducing the total processing delay by approximately 23%, as compared to the PPO algorithm.

Fig. 16 plots the UAV flight trajectory in the dynamic scenario. Compared to those in the static environment, the trajectories are more complex. The UAVs proactively adjust their directions in response to varying task requests. The figure also demonstrates that the proposed algorithm can continuously make optimal decisions based on the latest environmental information.

Simulation results show that the proposed algorithm exhibits superior performance and excellent robustness. Although the RL-based model requires extensive offline training, its online decision-making speed can reach a millisecond level. This completely avoids the time-consuming iterations required by traditional optimization methods, such as SCA.

## VII. Conclusion

In this article, we employed a dual time-scale framework to decompose the service caching and computation offloading problems into two sub-problems. We proposed a Lagrange dual decomposition-based method to solve the cache placement problem, and a self-attention mechanism enabled the MAPPO algorithm to design the UAV flight trajectory. To validate the performance of our proposed algorithms, we conducted extensive experiments. Long-term experiments demonstrate that our proposed algorithm achieves the performance improvements of 76.5% in CHR and 66% in system caching utility, as compared to the GSP algorithm. For the short-term optimization, our algorithm demonstrates a faster convergence speed and higher rewards when compared to the PPO policy. In disaster rescue scenarios, this framework can provide reliable and continuous computing services when terrestrial infrastructure does not work. By predicting the needs of the personnel to be rescued and dynamically adjusting UAV positions, the system significantly reduces the response and processing delays for time-sensitive missions. In our future work, it is necessary to design the backhaul resource allocation and consider the positive impacts of some influencing factors on the refined model, such as the service dependency.

## References


[1] G. Sun et al., "Joint task offloading and resource allocation in aerial-terrestrial UAV networks with edge and fog computing for post-disaster rescue," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8582–8600, Jan. 2024.

[2] J. Li et al., "UAV-assisted microservice mobile edge computing architecture: Addressing post-disaster emergency medical rescue," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2635–2648, Aug. 2025.

[3] P. Consul, R. Tiwari, S. B. Khan, E. Albalawi, F. Asiri, and A. Almusharraf, "A web 3.0 approach for forensic-driven secure resource allocation and trajectory optimization in UAV-assisted MEC network," *IEEE Trans. Consum. Electron.*, vol. 71, no. 3, pp. 8226–8233, Aug. 2025.

[4] W. Huang and Z. Ding, "New insight for multi-user hybrid NOMA offloading strategies in MEC networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 2918–2923, Feb. 2024.

[5] W. Zhang, G. Zhang, and S. Mao, "Deep-reinforcement-learning-based joint caching and resources allocation for cooperative MEC," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 12203–12215, Apr. 2024.

[6] Q. Liu, H. Zhang, X. Zhang, and D. Yuan, "Joint service caching, communication and computing resource allocation in collaborative MEC systems: A DRL-based two-timescale approach," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 15493–15506, Oct. 2024.

[7] J. Zhang, Y. Shen, Y. Wang, X. Zhang, and J. Wang, "Dual-timescale resource allocation for collaborative service caching and computation offloading in IoT systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1735–1746, Feb. 2023.

[8] P. Consul, I. Budhiraja, and D. Garg, "Task offloading in AIoT-enabled UAV-assisted MEC network: A digital twin-empowered approach with FedRL," *IEEE Trans. Consum. Electron.*, vol. 71, no. 1, pp. 1945–1956, Feb. 2025.

[9] C. Zhou, J. Liu, K. Qu, M. Sheng, J. Li, and W. Zhuang, "Delayaware UAV computation offloading and communication assistance for post-disaster rescue," *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 19110–19125, Dec. 2024.

[10] Y. Wan, H. Yao, J. Liu, C. Sun, A. Ma, and Y. Zhong, "Low-light and infrared multimodal remote sensing in nighttime rescue mission: A review of anomaly detection methods," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5403518.

[11] N. Lin, X. Han, A. Hawbani, Y. Sun, Y. Guan, and L. Zhao, "Deep reinforcement learning-based dual-timescale service caching and computation offloading for multi-UAV assisted MEC systems," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 1, pp. 605–617, Feb. 2025.

[12] M. Yan, C. Aun Chan, A. F. Gygax, C. Li, A. Nirmalathas, and I. Chih-Lin, "Efficient generation of optimal UAV trajectories with uncertain obstacle avoidance in MEC networks," *IEEE Internet Things J.*, vol. 11, no. 23, pp. 38380–38392, Dec. 2024.

[13] L. Sun, Z. Liu, Z. Ning, J. Wang, and X. Fu, "Multi-agent Q-Net enhanced coevolutionary algorithm for resource allocation in emergency human–machine fusion UAV-MEC system," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 4473–4489, 2025.

[14] M. Yan, H. Guo, C. A. Chan, A. F. Gygax, C. Li, and I. Chih-Lin, "Semantic communication-enabled multi-access edge computing network resource optimization in the 6G era," *IEEE Wireless Commun.*, vol. 33, no. 2, pp. 83–91, Apr. 2026.

[15] Y. Zhang, Z. Kuang, Y. Feng, and F. Hou, "Task offloading and trajectory optimization for secure communications in dynamic user multi-UAV MEC systems," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14427–14440, Dec. 2024.

[16] S. Ma, Y. Zhang, L. Peng, C. Sun, B. Ding, and Y. Zhu, "OWRTDETR: A novel real-time transformer network for small-object detection in open-water search and rescue from UAV aerial imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 4205313.

[17] J. Xu, M. Zhu, X. Liu, X. Li, and Y. Yang, "Cost-aware heterogeneous service placement strategies for MEC-based uncrewed delivery," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 6, pp. 5419–5432, Dec. 2025.

[18] A. H. Kazmi, A. Staffolani, T. Zhang, C. Cabrera, and S. Clarke, "Dynamic service placement in edge computing: A comparative evaluation of nature-inspired algorithms," *IEEE Access*, vol. 13, pp. 2653–2670, 2025.

[19] O. Elgarhy, Y. Le Moullec, L. Reggiani, M. M. Azeem, T. Taleb, and M. M. Alam, "Towards optimal placement and runtime migration of time-sensitive services of connected and automated vehicles," *IEEE Open J. Veh. Technol.*, vol. 6, pp. 13–33, 2025.

[20] C. Liu, Y. Zhong, R. Wu, S. Ren, S. Du, and B. Guo, "Deep reinforcement learning based 3D-trajectory design and task offloading in UAV-enabled MEC system," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 3185–3195, Feb. 2025.

[21] M. Zhao, R. Zhang, Z. He, and K. Li, "Joint optimization of trajectory, offloading, caching, and migration for UAV-assisted MEC," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1981–1998, Mar. 2025.

[22] J. Du, H. Wang, C. Jiang, J. Simonjan, J. Wang, and M. Debbah, "Distributed AI-based secure communications in space-air-ground-sea integrated networks," *IEEE Commun. Mag.*, vol. 63, no. 7, pp. 48–55, Jul. 2025.

[23] H. Zhang, J. Du, C. Jiang, J. Wang, F. Bader, and M. Debbah, "Task offloading in UAV-assisted mobile cloud-edge computing networks: An AoP-aware HAPPO approach," *IEEE Trans. Veh. Technol.*, vol. 74, no. 9, pp. 14745–14759, Sep. 2025.

[24] H. Wang, J. Du, C. Jiang, J. Wang, M. Debbah, and Z. Han, "Graph-aware temporal encoder-based service migration and resource allocation in satellite networks," *IEEE Trans. Wireless Commun.*, vol. 25, pp. 8260–8276, Dec. 2026.

[25] X. Bao, F. Liu, F. Zhang, and K. Yang, "Joint trajectory and resource optimization for delay minimization of UAV-enabled NOMA-MEC system with LWPT," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 4669–4688, Dec. 2026.


Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.

[26] W. Sun, H. Wang, Z. Qin, and X. Guo, "Air–ground collaborative networking and transmission scheduling for opportunistic UAV-assisted data collection," *IEEE Internet Things J.*, vol. 13, no. 3, pp. 4931–4948, Feb. 2026.
[27] P. Wu, F. Xiao, C. Sha, and H. Huang, "Service-oriented segmented trajectory design for low-altitude UAV-assisted MEC networks," *IEEE Trans. Mobile Comput.*, vol. 25, no. 2, pp. 2027–2044, Feb. 2026.
[28] M. Ismail et al., "Line-of-sight-based coordinated channel resource allocation management in UAV-assisted vehicular ad hoc networks," *IEEE Access*, vol. 12, pp. 25245–25253, 2024.
[29] Y.-I. Park, D.-Y. Kim, and J.-W. Lee, "Hybrid offline–online UAV trajectory design and subchannel allocation in UAV relaying OFDMA networks," *IEEE Internet Things J.*, vol. 11, no. 15, pp. 26173–26190, Aug. 2024.
[30] T. Zhou, B. Wang, D. Qin, X. Nie, N. Jiang, and C. Li, "Secure collaborative computation offloading and resource allocation in cache-assisted ultradense IoT networks with multislope channels," *IEEE Internet Things J.*, vol. 12, no. 11, pp. 17828–17843, Jun. 2025.
[31] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.
[32] S. Lu et al., "Enhanced multi-stage optimization of dynamic QoS-aware service caching and updating in mobile edge computing," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 4, pp. 3355–3369, Aug. 2025.
[33] A. D. Jiménez-Narváez et al., "Predictive modeling for fetal health: A comparative study of PCA, LDA, and KPCA for dimensionality reduction," *IEEE Access*, vol. 13, pp. 59687–59703, 2025.
[34] A. Ahmad, A. Aldalbahi, F. Ahmad, and S. Ali, "Service caching in multi-tier fog radio access networks," *IEEE Access*, vol. 12, pp. 182483–182495, 2024.
[35] J. Zhao, X. Hu, M. Yang, W. Zhou, J. Zhu, and H. Li, "CTDS: Centralized teacher with decentralized student for multiagent reinforcement learning," *IEEE Trans. Games*, vol. 16, no. 1, pp. 140–150, Mar. 2024.
[36] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 5998–6008.
[37] A. Bura, D. Rengarajan, D. Kalathil, S. Shakkottai, and J.-F. Chamberland, "Learning to cache and caching to learn: Regret analysis of caching algorithms," *IEEE/ACM Trans. Netw.*, vol. 30, no. 1, pp. 18–31, Feb. 2022.
[38] T. Zong, C. Li, Y. Lei, G. Li, H. Cao, and Y. Liu, "Cocktail edge caching: Ride dynamic trends of content popularity with ensemble learning," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 208–219, Feb. 2023.
[39] H. Shen, B. Ma, Z. Zhou, and Z. Wang, "GSMDV-hop: A modularized and greedy-strategy-based framework for enhanced DV-hop localization accuracy and stability," *IEEE Sensors J.*, vol. 25, no. 14, pp. 27653–27661, Jul. 2025.
[40] Y. Qu et al., "Service provisioning for UAV-enabled mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3287–3305, Nov. 2021.
[41] Y. Zeng, G. Guo, S. Chen, Y. Qiang, and J. Liu, "Energy-efficient data collection from UAV in WSNs based on improved PSO algorithm," *IEEE Sensors J.*, vol. 24, no. 21, pp. 35762–35774, Nov. 2024.
[42] J.-Y. Wang, M.-Y. Wang, W. Zeng, and Z.-A. Zhong, "Research on the imperfect information game of four-player Mahjong based on mix-PPO," *IEEE Trans. Games*, vol. 17, no. 2, pp. 485–497, Jun. 2025.



**Xuewei Zhang** (Member, IEEE) received the Ph.D. degree in information and communication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2020.

She is currently an Associate Professor with the School of Communications and Information Engineering, Xi'an University of Posts and Telecommunications, Xi'an, China. Her research interests include wireless edge caching and computing, physical layer security, advanced transmission design, and resource allocation.



**Jiantao Li** received the B.E. degree in electronic information engineering from Hebei Geo University, Shijiazhuang, China, in 2024. He is currently pursuing the M.S. degree in communication engineering with Xi'an University of Posts and Telecommunications, Xi'an, China.

His research interests include edge caching and UAVs.



**Yuan Ren** (Member, IEEE) received the B.Eng. degree in information engineering and the Ph.D. degree in signal and information processing from Beijing University of Posts and Telecommunications, Beijing, China, in 2010 and 2017, respectively.

He is currently an Associate Professor with Shaanxi Key Laboratory of Information Communication Network and Security, School of Communications and Information Engineering, Xi'an University of Posts and Telecommunications, Xi'an, China. His research interests include green communications, interference management, edge caching and computing, and cooperative communications.



**Fan Jiang** received the Ph.D. degree from the Key Laboratory of Universal Wireless Communication, Ministry of Education, Beijing University of Posts and Telecommunications, Beijing, China, in 2010.

She is currently a Full Professor at Xi'an University of Posts and Telecommunications, Xi'an, China. Her major research interests include wireless communication systems, such as device-to-device communications, fog computing/edge caching, and heterogeneous networks.



**Junxuan Wang** received the B.S. degree from Northwestern Polytechnical University, Xi'an, Shaanxi, China, in 1994, the M.E. degree from Xi'an University of Science and Technology, Xi'an, in 2002, and the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2005.

He is currently a Full Professor with the School of Communication and Information Engineering, Xi'an University of Posts and Telecommunications, Xi'an. His research interests include 5G/6G networks and wireless communications.



**Jie Zeng** (Senior Member, IEEE) received the B.S. and M.S. degrees from Tsinghua University, Beijing, China, in 2006 and 2009, respectively, the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, in 2019, and the Ph.D. degree from the University of Technology Sydney in 2021, Sydney, NSW, Australia.

From July 2009 to May 2020, he was with the Research Institute of Information Technology, Tsinghua University. From May 2020 to April 2022, he was a Post-Doctoral Researcher with the Department of Electronic Engineering, Tsinghua University. Since May 2022, he has been an Associate Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing. He has published over 100 journal and conference papers, and holds more than 40 Chinese and international patents. He participated in drafting one national standard and one communication industry standard in China. His research interests include 5G/6G, URLLC, satellite Internet, and novel network architecture.

Dr. Zeng received Beijing's Science and Technology Award in 2015, the Best Cooperation Award of Samsung Electronics in 2016, and Dolby Australia's Best Scientific Paper Award in 2020.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 07:05:35 UTC from IEEE Xplore. Restrictions apply.